TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT
ON MULTI-AGENT DATA ANALYSIS AND INSIGHT GENERATION PLATFORM
FOR FINANCIAL AND SALES DATA

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

August, 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON MULTI-AGENT DATA ANALYSIS AND INSIGHT GENERATION PLATFORM

FOR FINANCIAL AND SALES DATA

Submitted By:

Prashant Kafle (THA080BCT026)

Roshan Poudel (THA080BCT036)

Santosh Khadka (THA080BCT039)

Sushil Bhatta (THA080BCT048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the Bachelor's Degree in Computer Engineering.

Under the Supervision of

Er. Rajad Shakya

August, 2026

DECLARATION

We declare that the project report entitled **"Multi-Agent Data Analysis and Insight Generation Platform for Financial and Sales Data"**, submitted to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus** in partial fulfillment of the requirements for the Bachelor of Engineering degree in **Computer Engineering**, is a bonafide record of our work. The material in this report has not been submitted to any university or institution for the award of any degree, and no sources other than those listed here have been used.

Prashant Kafle (THA080BCT026)
Roshan Poudel (THA080BCT036)
Santosh Khadka (THA080BCT039)
Sushil Bhatta (THA080BCT048)

Date: August, 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus** the project work entitled **"Multi-Agent Data Analysis and Insight Generation Platform for Financial and Sales Data"**, submitted by **Prashant Kafle (THA080BCT026), Roshan Poudel (THA080BCT036), Santosh Khadka (THA080BCT039), and Sushil Bhatta (THA080BCT048)** in partial fulfillment of the requirements for the Bachelor's Degree in **Computer Engineering**. The project was carried out under supervision and within the time frame prescribed by the syllabus.

We found the students hardworking and skilled, and we recommend the acceptance of this work for partial fulfillment of the Bachelor's Degree in Computer Engineering.

Project Supervisor
Er. Rajad Shakya
Department of Electronics and Computer Engineering, Thapathali Campus

External Examiner

Project Co-ordinator
Asst. Prof. Anup Shrestha
Department of Electronics and Computer Engineering, Thapathali Campus

Head of the Department

Department of Electronics and Computer Engineering, Thapathali Campus

August, 2026

COPYRIGHT

ACKNOWLEDGEMENT

We thank the Department of Electronics and Computer Engineering, Thapathali Campus, Institute of Engineering, Tribhuvan University, for giving us the opportunity to work on this project.

We are grateful to our supervisor, Er. Rajad Shakya, for his guidance, constructive feedback, and encouragement throughout the project. His suggestions helped us refine both the technical design and the report structure.

We also thank the faculty members of the department for supporting practical engineering work. We are grateful to our friends and families for their patience and support during this phase of the project.

Submitted By:
Prashant Kafle (THA080BCT026)
Roshan Poudel (THA080BCT036)
Santosh Khadka (THA080BCT039)
Sushil Bhatta (THA080BCT048)

August, 2026

ABSTRACT

This project presents a multi-agent system that converts raw CSV-based financial and sales data into validated analytical reports with interactive charts and grounded plain-language insights. The system uses a LangGraph pipeline with six specialized agents: structural profiling, semantic tagging, context-aware preprocessing, statistical analysis and visualization, deterministic output validation with a Cohen's kappa check, and hybrid report generation with claim grounding. A FastAPI backend handles file uploads, job management with PostgreSQL persistence, and Server-Sent Events (SSE) for live progress updates. A React 19 frontend lets users upload datasets, monitor agent execution, view interactive charts, download HTML/PDF reports, and query data through retrieval-augmented generation (RAG) chat. The system accepts arbitrary tabular CSV files of financial and sales data; for demonstration, the report processes a sample sales dataset through all six agents and produces KPI summaries, correlation matrices, trend analyses, anomaly detection, and validated findings. Auditability is maintained through logged transformations, confidence propagation, Tier-1 contract checks, and LLM-to-heuristic agreement scoring. The result is a practical pipeline that combines deterministic data processing with controlled LLM reasoning for small to medium organizations.

Keywords: agentic frameworks, Cohen's kappa, FastAPI, financial analytics, LangGraph, multi-agent systems, RAG, React, reporting, sales analytics, validation

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BI	Business Intelligence
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DOECE	Department of Electronics and Computer Engineering
ECharts	Enterprise Charts
GQA	Grouped-Query Attention
IOE	Institute of Engineering
IQR	Interquartile Range
JSON	JavaScript Object Notation
JWT	JSON Web Token
KS	Kolmogorov-Smirnov
KV	Key-Value (cache)
LLM	Large Language Model
LPU	Language Processing Unit
MAD	Median Absolute Deviation
MAPE	Mean Absolute Percentage Error
MoM	Month-over-Month
OLS	Ordinary Least Squares
ORM	Object-Relational Mapping
PDF	Portable Document Format
QoQ	Quarter-over-Quarter
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SSE	Server-Sent Events
TU	Tribhuvan University
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1. Background

Data-driven decision making is now part of work in health, agriculture, education, and commerce. In Nepal, the government's Digital Nepal Framework gives priority to data-centric governance across these sectors. In practice, many organizations, especially outside the Kathmandu Valley, still manage spreadsheets manually and leave much of their data unused. Modern analytics tools also create a high entry barrier because users often need query languages, schema design, or statistical software skills. Enterprise platforms can also be too expensive for donor-funded non-governmental organizations (NGOs) and municipal bodies. Large Language Models (LLMs) and agentic orchestration frameworks now make a more automated approach possible [1]. LLMs can interpret column semantics and explain statistical findings in natural language, while frameworks such as LangGraph support reliable, stateful, multi-step workflows. Together, these tools can automate much of a human analyst's workflow: validation, cleaning, profiling, visualization, statistical testing, interpretation, and reporting.

1.2. Motivation

This project is motivated by the following problems:

- Despite the availability of data, many organizations lack the technical expertise required to analyze and interpret it effectively.
- Traditional data analysis tools often require statistical knowledge, programming skills, or manual report interpretation.
- Existing LLM-based systems may generate unverified or hallucinated insights, leading to unreliable conclusions.
- Recent advances in Artificial Intelligence and Multi-Agent Systems provide an opportunity to automate complex analytical tasks.

1.3. Problem Definition

Organizations accumulate data every day, including sales records, transaction logs, expense sheets, and customer information. Many still cannot convert that data efficiently into insights for strategy and decision making.

Existing challenges:

1. **Manual workflows:** Data analysis currently requires skilled staff to manually clean data, calculate metrics, build charts, and compile reports a process taking days to weeks

2. **Lack of standardization:** Without a systematic methodology, different people produce inconsistent results, making it difficult to compare periods or validate findings

3. **Limited accessibility:** Analytics tools require specialized knowledge. Non-technical stakeholders cannot ask ad-hoc questions without analyst intermediaries

This project proposes an automated system that takes raw sales and financial CSV data and produces validated, auditable reports with trend insights and recommendations.

1.4. Objectives

The primary objectives of this project are:

1. Develop an autonomous multi-agent system that converts raw sales and financial data into professional analytical reports, trend summaries, and recommendations without manual coding

2. Make basic financial analytics and sales optimization accessible to small and medium businesses through a web interface that does not require coding

1.5. Scope and Applications

Technical Scope:

- Supports common business data types such as sales records, transaction logs, and expense sheets. The web upload route currently accepts CSV files, while the internal Agent 1 loader also supports Excel, JSON/JSONL, and Parquet inputs when invoked directly.

- A six-agent pipeline (profiling, semantic tagging, preprocessing, statistical analysis, validation, and report generation) is implemented from input to report output

- Runs as a web application with FastAPI backend and React frontend, without requiring users to download additional software

Primary Use Cases:

1. **Sales Performance Tracking:** Automated analysis of sales data to identify trends, top-performing products, seasonal patterns, and revenue drivers for planning

2. **Financial Reporting & Budgeting:** Generation of expense summaries, income vs. expenditure analysis, budget tracking, and monthly/quarterly financial reports for small to medium organizations

3. **Retail & E-commerce Analytics:** Customer purchase behavior, inventory turnover analysis, sales channel comparison, and category performance review

4. **Small Business Operations:** Cash flow trends, profit margin analysis, supplier/payment tracking, and operational dashboards without requiring accounting software expertise

5. **Financial Data for Non-Finance Teams:** Simplified reports that translate raw financial/sales data into plain-language insights for managers, project leads, and decision-makers

1.6. Report Organization

The rest of this report is organized as follows. Chapter 2 reviews related work on BI platforms, AI-assisted analytics, automated visualization, and multi-agent frameworks, then identifies the research gap addressed by this project. Chapter 3 covers requirements, feasibility, dataset analysis, and implementation status. Chapter 4 explains the Qwen3-27B language model and the LLM inference process used in the system. Chapter 5 describes the system architecture, including the LangGraph pipeline, shared state, reliability propagation, backend services, and frontend workflow. Chapter 6 explains the implementation of the six agents, the FastAPI backend, and the React frontend. Chapter 7 reports the results from a complete run on the sample dataset, including tables, charts, and validation metrics. Chapter 8 lists planned extensions. Chapter 9 summarizes the project's contributions. The appendices include technical details, cost estimates, the timeline, and sample outputs.

2. LITERATURE REVIEW

2.1. Traditional Data Analytics Platforms

Business Intelligence platforms such as Microsoft Power BI, Tableau, and Google Looker Studio are widely used for visualization, dashboards, and reporting. They offer data connectors, charting tools, and reporting features that help organizations track performance indicators and business processes. These platforms work well in large organizations with the budget and staff to configure them. They are harder to adopt in smaller organizations because users still need to understand schemas, prepare data, and design dashboards. Licensing and infrastructure costs can also be difficult for small companies, educational institutions, non-profit organizations, and municipal bodies. A further limitation is that traditional BI tools usually visualize results but do not automate interpretation. Dwivedi et al. [1] describe how large language models are changing data analytics through natural language interaction, automated reasoning, and decision support. Their work supports the move from static dashboards toward AI systems that can explain and recommend actions from data. This project addresses that gap through a multi-agent data analyst system.

2.2. AI-Assisted Analytics and Large Language Model-Based Data Analysis

LLMs make it possible to interact with data using natural language [1]. Tools such as ChatGPT with Code Interpreter and other assistant-style systems allow users to upload datasets and receive code, charts, statistical analysis, and plain text explanations [2]. This reduces the technical effort needed for basic data analysis.

Their main advantage is accessibility. Users can ask questions about a dataset without writing code, and recent research shows that LLMs can support tasks such as cleaning data, exploratory analysis, visualization, and report generation [3].

These systems also have limits. Results can vary with prompt wording, and many assistant-style tools operate as single-agent systems without a fixed workflow or independent checks. Generated code can contain errors or omit steps [4, 5]. These issues support the need for systems that separate analysis, verification, and reporting into different stages. Retrieval-augmented generation (RAG) also helps ground LLM responses in external knowledge sources, reducing hallucinations and improving factual accuracy [19]. That idea is used in this project through dataset chat and claim grounding.

2.3. Automated Data Visualization from Tabular/CSV Data

Several studies have examined automated visualization generation from structured datasets. Data2Vis [6] proposed a sequence-to-sequence neural architecture that translates datasets into Vega-Lite visualization specifications. VizML [7] used machine learning to learn visualization design choices from more than one million dataset-visualization

pairs. Draco [8] represented visualization design knowledge as logical constraints for recommending visual encodings. DeepEye [9] automated chart generation and ranking. These studies support the use of an analysis agent that selects visualizations from the structure and semantics of tabular data.

This work shows that visualization can be treated as a recommendation or generation task rather than a fully manual design task. That is relevant for CSV-based analysis workflows, where the system must choose bar charts, line charts, scatter plots, or other encodings based on the dataset. This project follows the same direction by using an agent that converts cleaned tabular outputs into report-ready visualizations.

2.4. Agentic AI Frameworks and Multi-Agent Systems

Agentic AI frameworks such as LangGraph, CrewAI, and AutoGen allow multiple agents to coordinate on larger tasks [10, 11]. Instead of relying on a single assistant to perform every step, these frameworks assign separate responsibilities to different agents.

LangGraph supports stateful workflows with conditional routing and checkpointing. AutoGen focuses on agent-to-agent conversation, while CrewAI organizes task delegation through roles. These tools are useful when a system needs planning, coordination, and repeatable execution.

Wu et al. [11] show that structured multi-agent conversations can improve performance on complex tasks, including coding, mathematics, question answering, and decision support. Their work suggests that specialized agents can outperform a single general agent when the workflow is well defined.

Most existing work applies these frameworks to software engineering, web search, Q&A, and general reasoning. Less attention has been given to fixed data analysis workflows that combine deterministic statistical operations with language-model reasoning. This project applies multi-agent orchestration to automated data analysis and business intelligence reporting.

2.5. Existing Research on Automated Data Analysis Systems

Researchers have explored artificial intelligence methods for making data analytics easier to use. These systems combine machine learning, large language models, and natural language processing to support data exploration, visualization, and interpretation.

Sun et al. proposed LAMBDA, a Large Model Based Data Agent that uses specialized AI agents for data analysis [3]. The agents follow natural language instructions for tasks such as code generation, statistical analysis, and visualization. Their results show that coordinated agents can improve accuracy and dependability compared with single-agent approaches [3].

Even with this progress, many systems cover only part of the analytics process. Statistical tools are powerful but difficult for non-technical users. LLM tools are more accessible, but their outputs can change with prompt wording. Many tools also work in

isolation rather than covering the full process from preprocessing to report generation.

This leaves room for a multi-agent system that can provide reliable and repeatable data analysis across the full workflow.

2.6. Research Gap and Project Positioning

The reviewed literature shows a gap in multi-agent data processing systems that combine deterministic analytical algorithms with LLM-based reasoning in one reliable analytics pipeline [3, 11]. Traditional BI tools require user involvement and technical skill [1]. LLM-based systems improve accessibility, but they do not automatically provide reproducibility, auditability, or structured execution [4, 5]. Multi-agent frameworks provide orchestration, but they are used more often for software engineering and general reasoning than for autonomous data analytics and reporting [11, 10]. This project proposes a LangGraph-based multi-agent analytics system in which specialized agents perform data ingestion, preprocessing, analysis, visualization, validation, and report generation. Deterministic methods handle statistical computation, while LLM-based stages generate schema meaning and explanations. The goal is to improve reproducibility, accessibility, and automation for educational institutions, small businesses, and non-technical users.

3. REQUIREMENT ANALYSIS

3.1. Functional Requirements

- Upload or load a structured dataset.
- Profile the dataset structure and missingness.
- Infer semantic metadata for columns.
- Preprocess and clean the data.
- Generate statistical summaries and visualizations.
- Validate the outputs before final reporting.
- Produce a final executive report.

3.2. Non-Functional Requirements

- Reproducible execution through shared state and deterministic processing.
- Extensible agent-based design.
- Graceful handling of missing data, invalid input, and validation failures.
- Clear audit trail for preprocessing and validation decisions.

Traceability is the main non-functional requirement. The system keeps intermediate outputs in shared state and logs major actions as the pipeline runs, so decisions can be reviewed after execution. Fault tolerance is also required because the report generator must handle incomplete or imperfect inputs without producing unchecked results.

3.3. Software Requirements

The system is built on a Python backend with a React frontend. Main software components include:

- **Python 3.12+**: Core programming language for the backend and agents.
- **FastAPI**: Web framework for REST API endpoints and SSE streaming.
- **LangGraph**: Orchestration framework for building the six-agent pipeline with conditional routing.
- **Pandas, NumPy, SciPy, Scikit-learn**: Data manipulation, statistical analysis, and machine learning utilities.

- **Matplotlib, ECharts**: Static and interactive chart generation.
- **Jinja2, WeasyPrint**: HTML templating and PDF rendering.
- **PostgreSQL with pgvector**: Optional persistent storage for job data and RAG embeddings.
- **React 19, Vite, TailwindCSS**: Frontend framework, build tool, and styling.

3.4. Feasibility Study

3.4.1. Technical Feasibility

The project uses mature open-source libraries and cloud-based LLM APIs. Python libraries such as Pandas, Scikit-learn, and LangGraph provide the data processing and orchestration foundation. Groq and Gemini APIs remove the need for local GPU infrastructure, so the system can run on standard cloud VMs or local machines with internet access. The React frontend runs in the browser, which supports cross-platform use.

3.4.2. Operational Feasibility

The web interface lowers the entry barrier. Users can upload a CSV file and receive an analytical report without writing code or using statistical software. The system is designed for self-service use, reducing dependence on specialized analysts.

3.4.3. Economic Feasibility

The main variable cost is LLM API usage. Deterministic fallback mechanisms and heuristic tagging reduce the need to call the LLM for every operation. The core stack, including Python, FastAPI, React, and PostgreSQL, is open source, so there are no software licensing fees for those components. For small to medium organizations, API usage cost is expected to be lower than the labor cost of repeated manual analysis.

3.5. Dataset Analysis

The system accepts arbitrary tabular CSV files of financial and sales data. For validation and demonstration, the repository includes `10000 Sales Records.csv`. This sample contains 10,000 rows and 14 columns representing simulated global sales transactions. Its columns are:

- `Region`, `Country`: Geographic fields (categorical).
- `Item Type`, `Sales Channel`, `Order Priority`: Categorical labels (12, 2, and 4 distinct values respectively).
- `Order Date`, `Ship Date`: Transaction and shipment dates (datetime).

- `Order ID`: Unique transaction identifier.
- `Units Sold`: Quantity sold (count).
- `Unit Price`, `Unit Cost`, `Total Revenue`, `Total Cost`, `Total Profit`: Monetary fields (currency).

The sample's mix of identifiers, dates, currencies, and categories makes it suitable for exercising the full pipeline. Agent 2's semantic tagging produces a blueprint in which `Order ID` is flagged as an identifier, `Order Date` and `Ship Date` as datetime, `Region` and `Country` as geographic, and the five monetary columns as currency. The preprocessing stage applies financial constraints, extracts date features, one-hot encodes categorical columns, and derives business metrics such as profit margin and revenue per unit.

3.6. Requirement Status

Table 3-1. Implementation status of system requirements

Requirement	Status	Notes
Data loading and profiling	Implemented	Agent loader supports CSV/TSV-style text, Excel, JSON/JSONL, and Parquet; web upload currently accepts CSV
Semantic tagging	Implemented	Qwen3-27B with Gemini fallback
Preprocessing and cleaning	Implemented	Audit trail, quality scoring, KNN/iterative imputation
Statistics and charts	Implemented	ECharts + matplotlib, 10+ chart types
Validation gate	Implemented	Tier-1 contract checks + Cohen's kappa gate
Final report generation	Implemented	HTML/PDF via Jinja2/WeasyPrint
Backend API	Implemented	FastAPI + SSE + PostgreSQL/File storage
Frontend dashboard	Implemented	React 19 + Vite + TailwindCSS
Multi-format expansion	Partial	Direct loader support exists; API upload remains CSV-only

4. THEORETICAL APPROACH

The semantic tagging stage uses a Transformer-based Large Language Model (LLM) through the Groq API with the `qwen/qwen3.6-27b` model. The system also includes an automatic Gemini fallback and a deterministic fallback when LLM calls fail. The following sections explain how the model produces the JSON schema blueprint that drives preprocessing.

4.1. Large Language Model Theory

A Large Language Model is a neural network trained to model the probability distribution of natural language. Given a sequence of previous tokens, it estimates the most likely next token. In this project, the model is used as a structured reasoning component, not as an unrestricted prose generator. It reads column metadata and returns a strict JSON schema blueprint.

4.1.1. Decoder-Only Transformer Architecture

Earlier sequence models such as RNNs and LSTMs processed tokens one step at a time, which made training slow and long-range relationships hard to capture. The Transformer replaces recurrence with a self-attention mechanism that relates every token to every other token in parallel [12]. Text-generation models such as the Qwen family use a *decoder-only* Transformer: a stack of identical blocks in which a causal mask allows the prediction at position t to attend only to positions 1 to t. This left-to-right constraint makes the model autoregressive, generating one token at a time.

4.1.2. Tokenization and Embedding

Raw text is first split into subword tokens, each mapped to an integer index in a fixed vocabulary of size V. Every index is then converted into a dense vector through a learned embedding matrix $E \in \mathbb{R}^{V \times d}$, where d is the model dimension:

$$x = E(\text{token}) \tag{4.1}$$

Because self-attention is order-agnostic, a positional encoding is added to each embedding so the model can distinguish tokens by position as well as identity. In this project, the prompt containing column names, inferred types, and sample values is tokenized and embedded in exactly this way before inference.

4.1.3. Self-Attention

Self-attention lets each token gather information from other tokens. Three projections are formed from the input: queries Q, keys K, and values V, and they are combined

using scaled dot-product attention:

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{4.2}$$

where d_k is the key dimension and $\sqrt{d_k}$ scales the dot products to keep the softmax stable [12]. The Transformer runs several attention *heads* in parallel so the model can capture different relationships at once. Each block also applies a position-wise feed-forward network with residual connections and layer normalization, which add capacity and keep the deep computation numerically stable.

4.1.4. Next-Token Prediction

After the final block, the last token representation is projected to a vector of logits z of length V, which a softmax converts into a probability distribution over the vocabulary:

$$P(y_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{4.3}$$

The model predicts the next token conditioned on all previous tokens and generates text autoregressively: each predicted token is appended to the sequence and fed back in to predict the following one, until a stopping condition is reached. A low sampling temperature is used in this project so the output is nearly deterministic, which is important because the response must be strictly valid JSON.

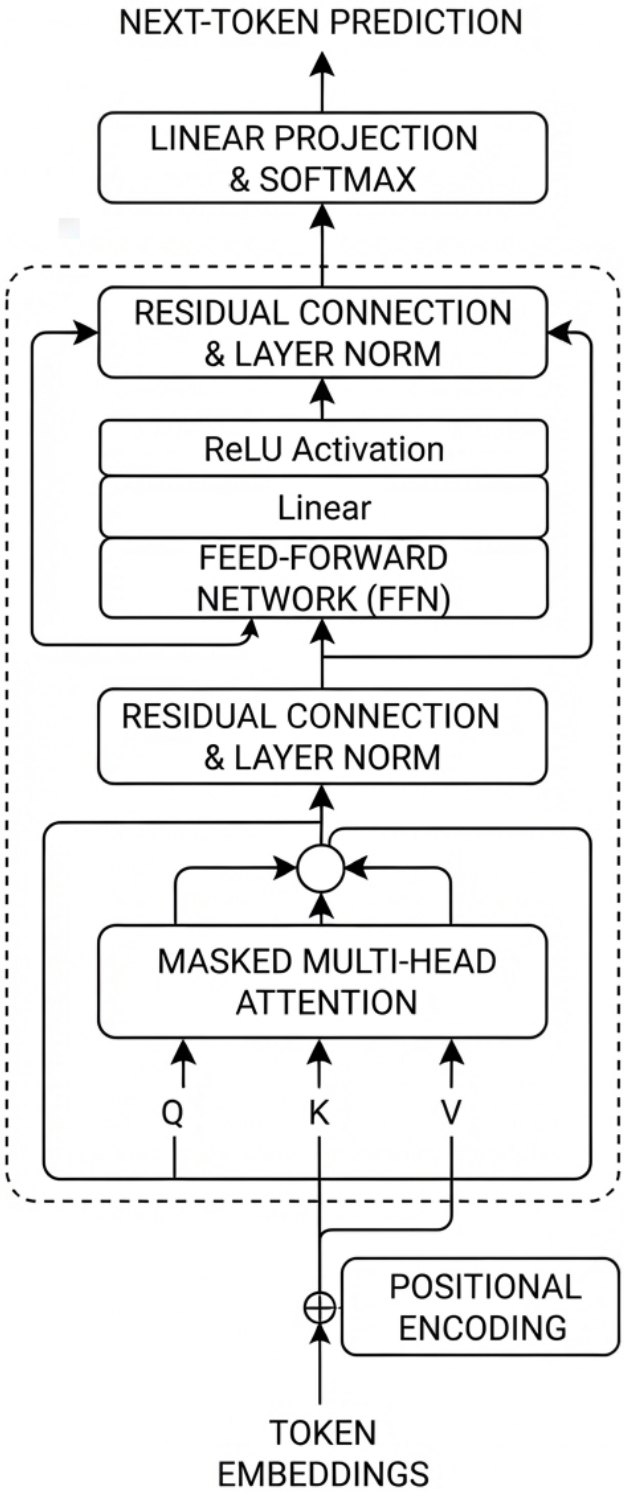

Figure 4-1. Decoder-only Transformer architecture used by modern LLMs

4.2. Groq API and LLM Inference Architecture

The LLM backend uses the Groq API, which exposes the `qwen/qwen3.6-27b` model through an OpenAI-compatible REST interface. Groq runs Transformer models on Language Processing Units (LPUs), custom silicon designed for autoregressive token gener-

ation. The system also includes an automatic Gemini fallback (`gemini-flash-latest` and 3.x variants) and a deterministic fallback when the primary model is unavailable. This section explains the inference model and how Agent 2 uses it to generate the JSON schema blueprint.

4.2.1. Why the Groq API Is Used

The Groq API was selected over local inference runtimes for three reasons.

Inference speed. LPU hardware is designed for the sequential, memory-bound nature of autoregressive decoding. Each forward pass generates one token, and the dominant cost is reading model weights from memory on every step. LPUs use on-chip SRAM instead of off-chip DRAM, reducing the memory bandwidth bottleneck. For a pipeline where Agent 2 may need to tag dozens of columns in one call, short round-trip times help keep overall pipeline latency acceptable.

No local hardware requirement. Running a multi-billion-parameter model locally demands tens of gigabytes of GPU VRAM. The Groq API allows the project to use an open-weight model without requiring a local GPU. This makes the system usable on standard laptops and cloud VMs.

Simplicity and cost control. The API follows the same chat-completion contract as the OpenAI SDK. Agent 2 constructs a system prompt and a user message, sends a single HTTP POST request, and receives a JSON response. The official Python client (`groq`) handles authentication and retries. Usage-based pricing avoids fixed model-hosting infrastructure costs. Agent 2 sends only column metadata and three sample values rather than full table rows, which limits prompt size by design.

4.2.2. The Qwen3-27B Model

`qwen/qwen3.6-27b` is a multi-billion-parameter decoder-only Transformer released by the Qwen team. Its architecture follows the design described in Section 4.1 with several refinements that improve efficiency and long-context handling.

Grouped-Query Attention (GQA). Standard multi-head attention stores one key-value head per query head. Grouped-Query Attention partitions the query heads into groups that share a single key-value head. Formally, if there are H query heads organized into G groups ($G < H$), the attention for query head h in group g is:

$$\text{GQA}_h(Q_h, K_g, V_g) = \text{Softmax}\left(\frac{Q_h K_g^\top}{\sqrt{d_k}}\right) V_g \tag{4.4}$$

where K_g and V_g are the shared key and value matrices for group g [14]. GQA reduces the size of the KV cache proportionally to the group ratio H/G, which lowers memory bandwidth pressure during autoregressive decoding and helps LPU-based inference run efficiently with this model.

RoPE positional encoding. Instead of adding a fixed positional vector to each

embedding, Qwen applies Rotary Position Embedding (RoPE), which encodes absolute position by rotating the query and key vectors before the dot product:

$$\tilde{q}_m = q_m \cdot e^{im\theta}, \quad \tilde{k}_n = k_n \cdot e^{in\theta} \tag{4.5}$$

After rotation the dot product $\tilde{q}_m^\top \tilde{k}_n$ depends only on the relative position $m-n$, giving the model a principled notion of token distance [15]. Compared with learned absolute embeddings, RoPE generalizes more cleanly to prompt lengths not seen during training.

SwiGLU feed-forward blocks. Each Transformer block contains a position-wise feed-forward network. Qwen replaces the standard ReLU activation with SwiGLU:

$$\mathrm{FFN}(x) = (\mathrm{Swish}(xW_1) \odot xW_2)\, W_3 \tag{4.6}$$

where $\mathrm{Swish}(x) = x \cdot \sigma(x)$ is a smooth gating function, $\odot$ is element-wise multiplication, and W_1, W_2, W_3 are learned weight matrices [16]. The gating mechanism allows the network to suppress irrelevant dimensions adaptively, which empirically improves output quality over ReLU at the same parameter count.

RMS Layer Normalization. Qwen uses Root Mean Square Layer Normalization (RMSNorm) instead of the standard LayerNorm:

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}(x)} \cdot \gamma, \quad \mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2} \tag{4.7}$$

RMSNorm omits the mean-centering step of LayerNorm [17]. Because re-centering is often redundant given downstream weight matrices, RMSNorm achieves similar training stability at lower computational cost.

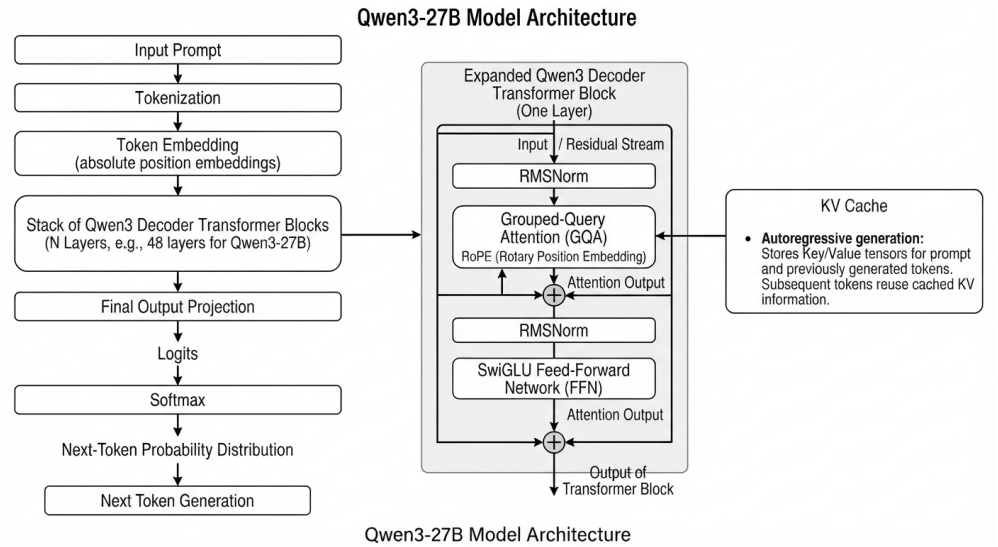

Figure 4-2. Qwen3-style decoder-only Transformer with GQA, RoPE, SwiGLU, and RMSNorm

4.2.3. Agent 2 API Call and Prompt Processing

Agent 2 calls the Groq API exactly once per tagging request (or per batch for wide schemas). The call flow is as follows.

1. **Prompt construction.** `_build_llm_prompt` assembles a *system* message that specifies the required JSON schema format and valid tag values, and a *user* message containing, for each column: the column name, the heuristically inferred type, the missing-value rate, and the first three sample values. This keeps the prompt limited to schema evidence rather than full table rows.

2. **API request.** The Groq Python client sends a `chat.completions.create` request with `model="qwen/qwen3.6-27b"` and `temperature=0.1`, `reasoning_effort="none"`. A temperature close to zero makes the softmax distribution (Equation 4.3) sharply peaked, so the model consistently selects the highest-probability token rather than sampling freely. This near-determinism is important because the output must be strictly parseable JSON.

3. **Prompt processing.** On Groq's LPU cluster the prompt tokens are tokenized and embedded (Equation 4.1), then passed through the decoder blocks of Qwen3-27B. Inside each block, GQA (Equation 4.4) computes position-aware attention using RoPE (Equation 4.5), and the SwiGLU FFN (Equation 4.6) applies non-linear gating. RMSNorm (Equation 4.7) stabilizes each sub-layer. The final block projects to logits over the vocabulary and applies softmax to produce the next-token distribution.

4. **KV cache and autoregressive generation.** After the prompt is processed, the key and value tensors for every prompt token are cached. Each subsequent output token is generated by running only the new token through the network, attending to the cached values. This KV-cache mechanism makes generation fast even for long prompts.

5. **Gemini failover.** If the Groq call fails or returns an unparseable response, the system automatically falls back to Google Gemini (`gemini-flash-latest`, then `gemini-3.6-flash`, `gemini-3.5-flash`, `gemini-2.5-flash`) through `_call_gemini_json_with_failover`. If all LLM providers fail, `_fallback_blueprint` constructs a conservative schema from the locally sniffed types, so the pipeline still has a usable blueprint.

6. **Response parsing.** The generated text is returned in the API response body. `_parse_schema_blueprint_response` extracts valid JSON from raw or code-fenced output; as a last resort it locates the outermost `{...}` span so that a stray

formatting character does not discard an otherwise correct schema. If parsing fails after retries, `_fallback_blueprint` constructs a conservative schema from the locally sniffed types, so the pipeline still has a usable blueprint.

Agent 2 is loosely coupled to the model provider. The LLM interaction is isolated inside a few functions (`_build_llm_prompt`, the Groq/Gemini client calls, and `_parse_schema_blueprint_response`), so another OpenAI-compatible endpoint could be used without changing the rest of the pipeline.

5. SYSTEM ARCHITECTURE AND METHODOLOGY

5.1. Overall Pipeline

The backend pipeline is built as a directed sequence of agents using LangGraph's `StateGraph`. Each agent reads the current `GraphState`, writes its outputs back into that state, and passes the result to the next step. This makes the workflow easier to inspect and debug. Conditional edges control whether execution continues, based on errors and validation results.

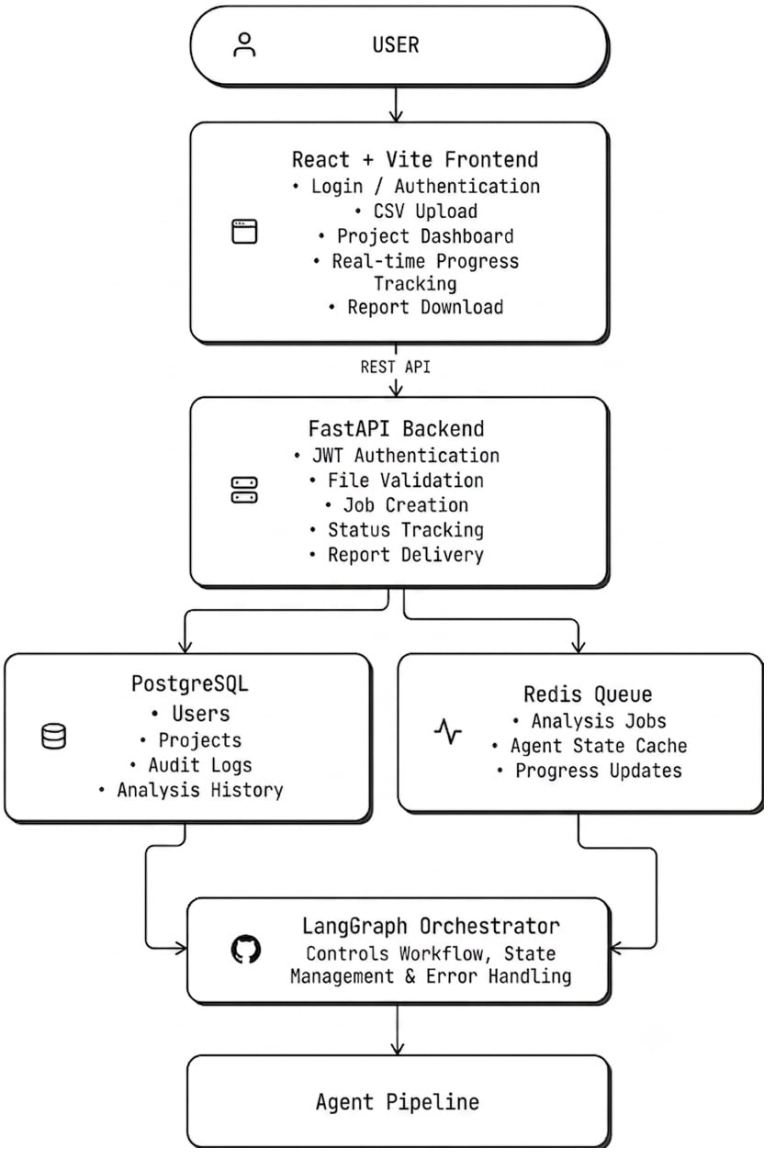

Figure 5-1. System architecture and agent flow (six-agent pipeline)

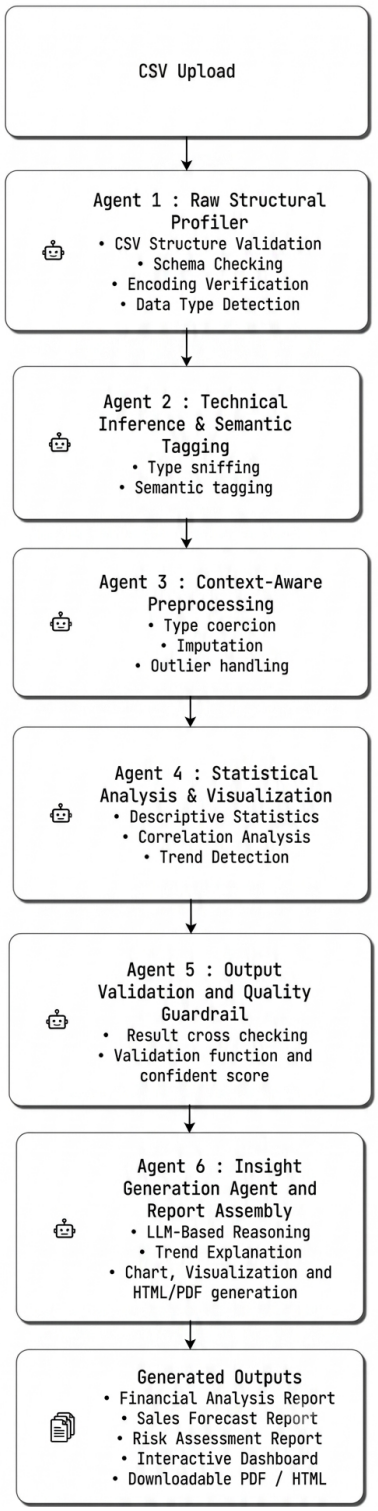

Figure 5-2. LangGraph workflow showing conditional routing and Agent 5 gate

5.2. Shared State Design

The shared state (`GraphState`) contains the input path, raw profile, semantic blueprint, cleaned dataset, scaling parameters, preprocessing log, quality score, statistics, chart paths, errors, and reliability metadata. This lets each agent focus on one responsibility

while the pipeline still keeps a single record of the run. The state also carries validation results, insight facts, narrative text, and the final report path.

Each stage contributes a confidence score and evidence list. These values help the system decide whether later stages should proceed and let the final report show how reliable the outputs are.

5.3. LangGraph Orchestration and Conditional Routing

The pipeline is built with `langgraph.graph.StateGraph(GraphState)`. Nodes are `agent1`..`agent6`, with entry point `agent1`. Conditional edges gate the flow:

- After **Agent 1**: end if any error containing "Agent1" or no `raw_profile`, else → Agent 2.
- After **Agent 2**: end if "Agent2" error or no `schema_blueprint`, else → Agent 3.
- After **Agent 3**: end only if `cleaned_df is None` (non-fatal "Agent3:" warnings do not abort), else → Agent 4.
- After **Agent 4**: end if "Agent4" error, else → Agent 5.
- After **Agent 5**: end if "Agent5" error **or** `validation_report["passed"]` is falsy. Agent 5 is a hard quality gate: a failed validation stops the pipeline before reporting, else → Agent 6.
- **Agent 6** → END.

5.4. Reliability and Confidence Propagation

The reliability model stores each stage's confidence in $[0,1]$ under `reliability.stage_confidence[stage]`. Overall confidence is the mean of the available stage confidences, and decision readiness is classified as `ready` (≥ 0.85), `needs_review` (≥ 0.65), or `blocked` (< 0.65). Evidence strings accumulate throughout the pipeline so the final result can be audited. The overall reliability is:

$$\bar{c} = \frac{1}{N} \sum_{i=1}^{N} c_i \tag{5.1}$$

where c_i is the confidence from agent i. This mechanism is implemented in `update_reliability` in `backend/main.py`.

5.5. Backend Architecture

The FastAPI backend provides REST endpoints for authentication, file upload, job management, SSE progress streaming, report download, and grounded RAG dataset chat.

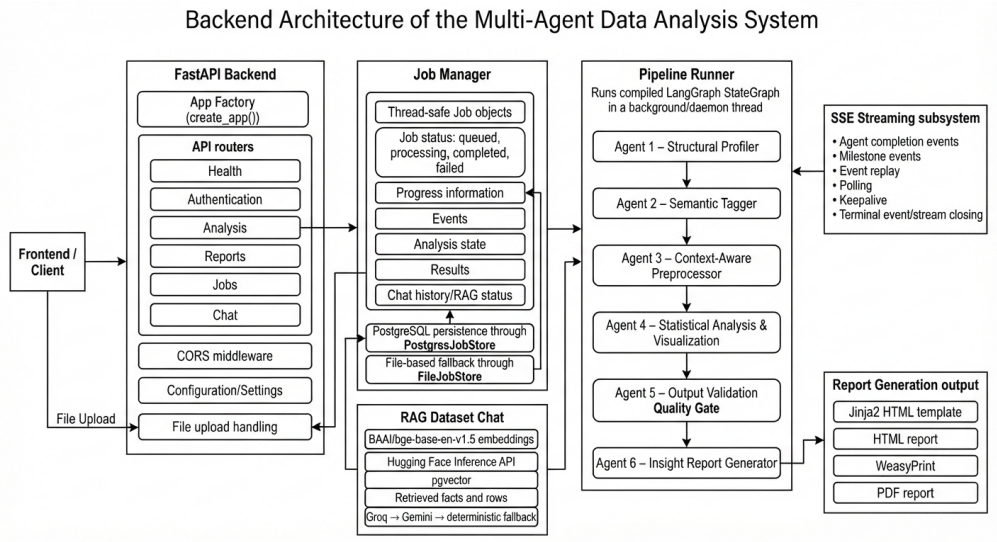

Figure 5-3. Backend architecture: FastAPI, job manager, pipeline runner and RAG components

5.5.1. Core Components

- **App Factory:** `create_app()` initializes routers, CORS, and global exception handlers.
- **Job Manager:** Thread-safe `Job` dataclass with status, progress, events, and optional PostgreSQL (`PostgresJobStore`) or file-based (`FileJobStore`) persistence.
- **Pipeline Runner:** Runs compiled LangGraph in a daemon thread, emitting SSE events via `_MilestoneEmitter` at each agent completion and milestone.
- **SSE:** `/api/analyze/{job_id}/stream` provides real-time progress updates with polling and keepalive.
- **RAG Chat:** Uses `BAAI/bge-base-en-v1.5` embeddings via Hugging Face Inference API, stores embeddings in pgvector, and grounds answers in retrieved facts and rows.

5.5.2. API Endpoints

Table 5-1 summarizes the main endpoints.

Table 5-1. FastAPI backend endpoints

Endpoint	Method	Description
`/api/health`	GET	Health check
`/api/auth/signup`	POST	User registration with PBKDF2-SHA256 hashing
`/api/auth/login`	POST	User login, returns JWT-style token
`/api/analyze`	POST	Upload CSV, start analysis job, returns job ID
`/api/analyze/{id}/stream`	GET	SSE progress stream
`/api/analyze/{id}/result`	GET	AnalysisResult (charts, stats, report path)
`/api/jobs`	GET	List past jobs
`/api/jobs/{id}`	GET	Job summary
`/api/report/{id}`	GET	Download PDF/HTML report
`/api/analyze/{id}/chat`	GET/POST	Dataset Q&A with RAG

5.6. Frontend Workflow

The React 19 frontend (`frontend/AnalyzeAI/`) provides a web interface for uploading datasets, monitoring agent progress, viewing results, and interacting with the data through RAG chat.

- **Stack:** React 19.2.7, react-router-dom 7.18.1, TailwindCSS 4.3.2, local shadcn-style UI primitives, framer-motion, lucide-react, and Vite 8.1.0.
- **Routing:** `/` (LandingPage), `/analyze` (AnalyzePage), `/analyze/:jobId` (result view), `/history` (HistoryPage), `/login`, `/signup`, `/profile`.
- **Authentication:** `AuthContext` stores user in localStorage; `RequireAuth` guards protected routes.
- **SSE Consumption:** `subscribeToJobStream` uses `EventSource` to listen for `csv_loaded`, `pipeline_started`, `agent_progress`, `charts_generated`, `validation_passed/failed`, `report_generated`, and `pipeline_finished/failed` events.
- **Results Display:** Displays agent progress with color coding (blue through rose), KPI cards, ECharts interactive charts (embedded JSON), and a grounded dataset chat panel.

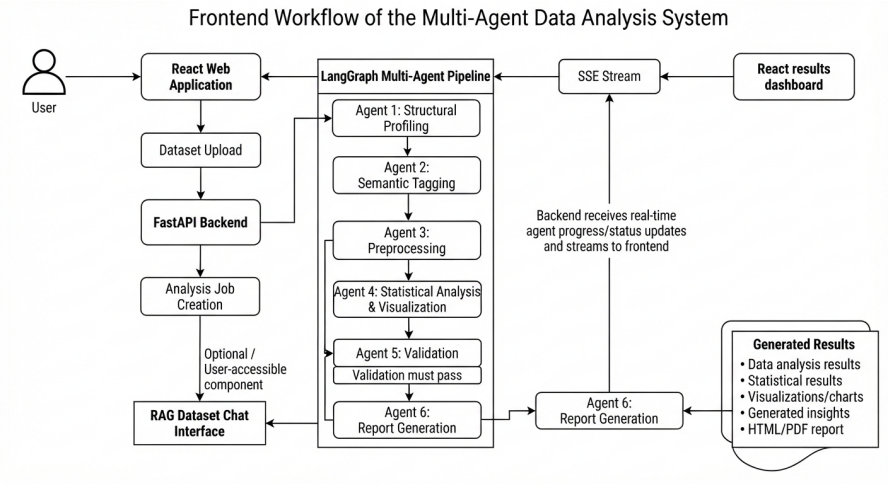

Figure 5-4. Frontend workflow: user upload, SSE progress, results, and RAG chat

6. IMPLEMENTATION DETAILS

6.1. Agent 1: Structural Profiler

6.1.1. Overview and Purpose

Agent 1 (`agent1_structural_profiler`) is the entry point of the pipeline. It reads the input file and records a factual description of its structure. It performs no cleaning or inference, so later stages work from an unmodified baseline.

6.1.2. Inputs and Outputs

- **Inputs:** `csv_path` from the shared `GraphState`, an optional `sheet_name` for Excel workbooks, and the running `errors` list.
- **Outputs:** `raw_profile` (per-column and dataset-level summary), the cached DataFrame `_df_cache`, updated `errors`, and stage `reliability` metadata.

6.1.3. Processing Workflow

The agent proceeds in four ordered stages, summarised in Figure 6-1:

1. `_load_dataframe` selects a reader based on the file extension, routing CSV/TSV-style text, Excel, JSON/JSONL, and Parquet files to dedicated loaders.
2. the chosen reader tolerates real-world messiness such as mixed delimiters and non-UTF encodings.
3. each column is measured for type, missingness, uniqueness, and sample values.
4. a confidence value is computed from the observed missingness and duplication.

Figure 6-1. Workflow of Agent 1: Structural Profiling

6.1.4. Main Functions

`_read_mixed_delimiter_csv`**: resilient CSV ingestion.** Business exports can mix comma- and semicolon-separated rows under non-UTF encodings. This function first reads the text through `_read_csv_lines` (trying `utf-8-sig`, `cp1252`, and `latin-1`), then detects the delimiter *per line*. If the field count does not match the header, it retries with the alternate delimiter before raising an error (Figure 6-2). A wrong delimiter would merge columns and corrupt later stages, so the parser fails explicitly when it cannot recover. The profiling step records each column's dtype, missing count and rate, unique count, and first three sample values, which Agent 2 uses instead of the full dataset.

```
for line in lines[1:]:
    if not line.strip():
        continue

    delimiter = ";" if line.count(";") > line.count(",") else ","
    fields = next(csv.reader([line], delimiter=delimiter, quotechar='"'))

    if len(fields) != len(header):
        alternate = "," if delimiter == ";" else ";"
        alt_fields = next(csv.reader([line], delimiter=alternate, quotechar='"'))
        if len(alt_fields) == len(header):
            fields = alt_fields
        else:
            raise ValueError(
                f"Unable to parse row with {len(fields)} fields "
                f"(expected {len(header)}): {line[:120]}"
            )

    writer.writerow(fields)
```

Figure 6-2. Per-line delimiter detection and fallback in `_read_mixed_delimiter_csv` (Agent 1)

6.1.5. Reliability and Pipeline Interaction

`_compute_agent1_confidence` converts the observed data health into a single confidence value that decreases with missingness and duplication:

$$c_1 = \text{clamp}\left(1 - 0.5\,\frac{m}{100} - 0.3\,\frac{d}{100},\ 0,\ 1\right) \tag{6.1}$$

where m is the overall missing rate and d the duplicate rate as percentages. The stage is marked `ready` when $c_1 \geq 0.8$ and `needs_review` otherwise. On success it writes `raw_profile`, caches the DataFrame in `_df_cache`, and passes both to Agent 2; if loading fails, the error is recorded and the pipeline stops early. Tested on the `10000 Sales Records.csv` dataset, Agent 1 parsed the file without row loss and produced the per-column missingness and duplicate counts consumed by the later agents.

6.2. Agent 2: Semantic Tagger

6.2.1. Overview and Purpose

Agent 2 (`agent2_semantic_tagger`) turns the structural profile into a `schema_blueprint`, which describes each column's intended type, semantic role, and cleaning policy. Agent 1 records what the data looks like; Agent 2 assigns meaning so preprocessing can follow rules rather than guesswork. When the language model is invoked, it is called through the Groq API (`qwen/qwen3.6-27b`); Chapter 4 describes that inference process.

6.2.2. Inputs and Outputs

- **Inputs:** `raw_profile` and the cached DataFrame `_df_cache` from Agent 1, plus the `errors` list.
- **Outputs:** the `schema_blueprint` (per-column type, semantic tag, identifier flag, scaling and imputation policy, encoding strategy, and null policy) with a `__metadata__` block and updated `errors`.

6.2.3. Processing Workflow

The agent combines cheap local heuristics with a controlled model call. The workflow is illustrated in three stages in Figures 6-3 to 6-5.

First, local type sniffing (`_infer_intended_types`) assigns a provisional type to every column, and a minimal prompt containing only column metadata and three sample values is built (Figure 6-3).

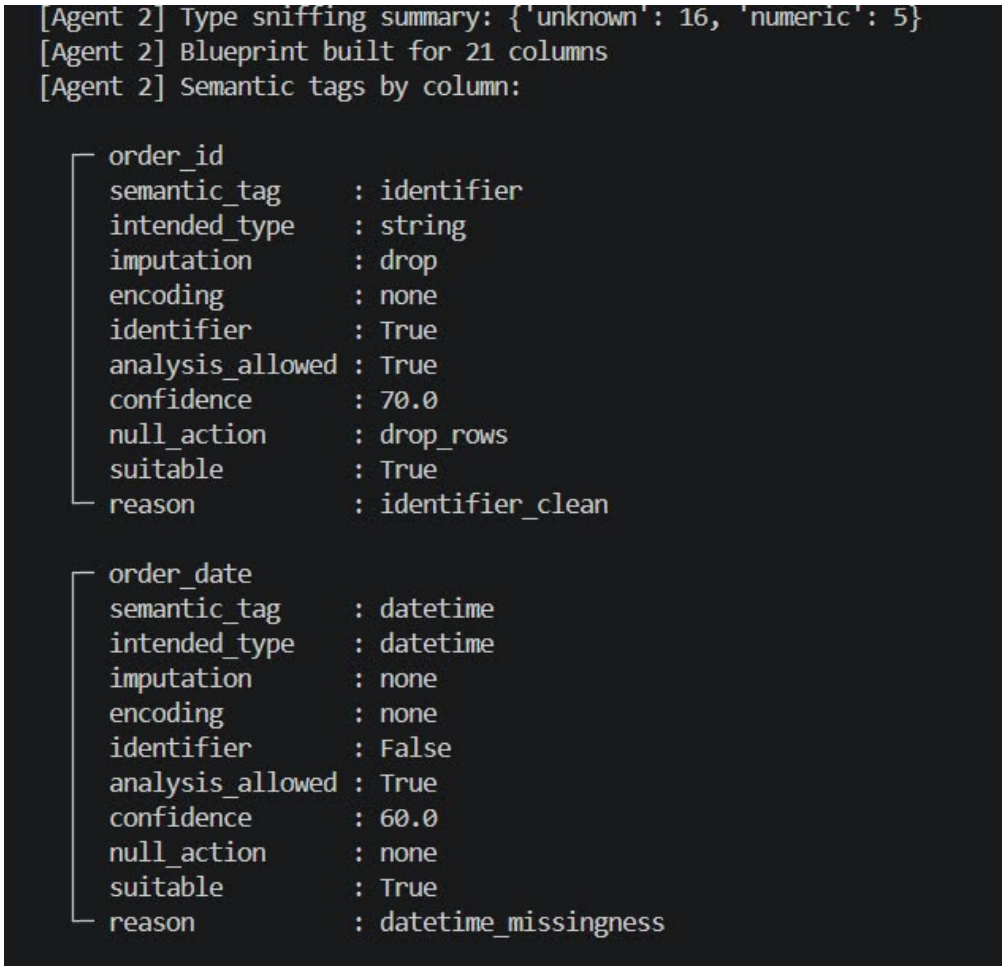

Figure 6-3. Agent 2 workflow (stage 1): local type inference and prompt construction

Next, the model is called once for small schemas or in retry-safe batches for wide datasets and its JSON response is parsed defensively into the schema blueprint (Figure 6-4).

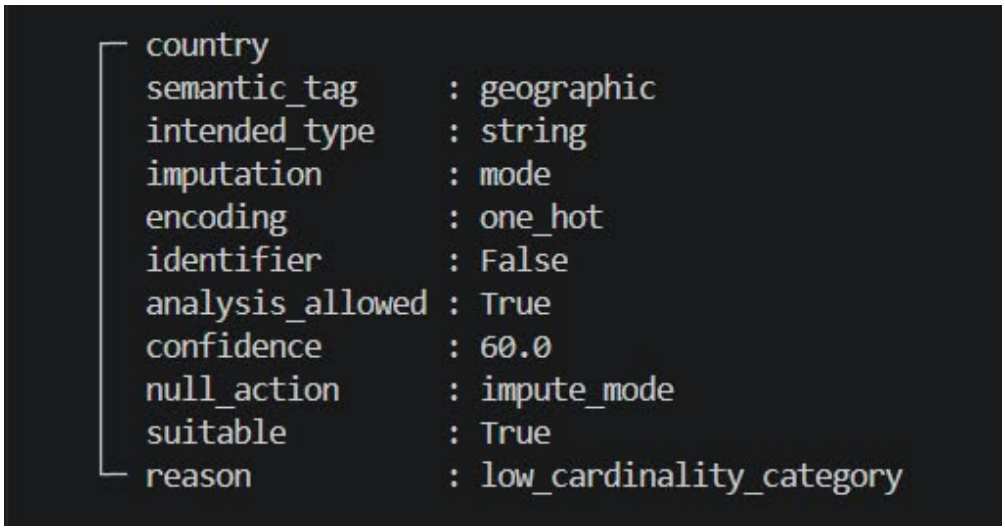

Figure 6-4. Agent 2 workflow (stage 2): batched LLM inference and schema parsing

Finally, the blueprint is enriched: missing tags are filled, a missingness policy is applied, per-column confidence is scored, and a conservative fallback is used if the model output is unusable (Figure 6-5).

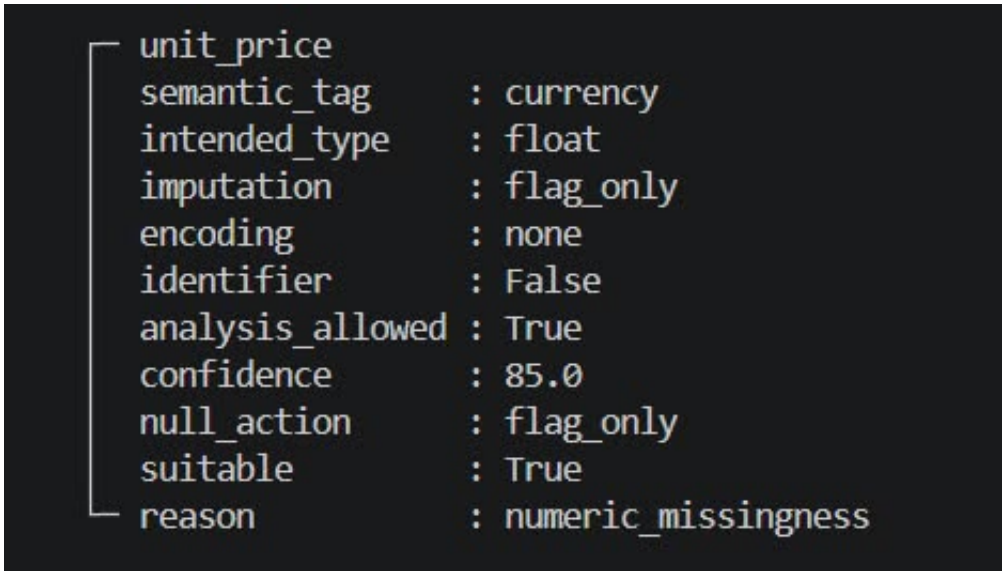

Figure 6-5. Agent 2 workflow (stage 3): enrichment, confidence scoring, and fallback

6.2.4. Main Functions

`_infer_intended_types` performs local type sniffing without the model, using the Pandas dtype and an 80% parseability threshold for object columns; its result seeds both the prompt and the fallback, so every column has a defensible type even when the LLM is unavailable. `_build_llm_prompt` sends only column metadata and three samples rather than the full table, and a fixed system prompt requires strictly valid JSON at a low temperature (0.1). For wide schemas, `_call_llm_for_schema_blueprint_with_retry` splits columns into batches and, if a truncated response fails to parse, recursively halves the batch and merges the results, which lets the agent tag very wide datasets reliably. `_parse_schema_blueprint_response` accepts raw or fenced JSON and, as a last resort, extracts the outermost `{...}` span, so a single formatting quirk does not discard a valid schema. `_call_gemini_json_with_failover` iterates through Gemini model candidates if the Groq call fails.

6.2.5. Reliability and Pipeline Interaction

After tagging, missing `semantic_tag` values are filled and each column receives a `null_policy` and `encoding_strategy`; columns whose missing rate exceeds 20% are marked `analysis_allowed = false` so later analysis ignores unreliable fields, and `_calculate_semantic_confidence` scores each tag from 0 to 100. If the model output is invalid, `_fallback_blueprint` assigns conservative policies from the sniffed types. On the sample `10000 Sales Records.csv` dataset, Agent 2 tagged the five

monetary columns (Unit Price, Unit Cost, Total Revenue, Total Cost, Total Profit) as currency, Units Sold as a count, Region and Country as geographic, Item Type/Sales Channel/Order Priority as categorical labels, Order Date and Ship Date as datetime, and Order ID as an identifier. Example tags: `Order ID` → identifier/conf 80; `Order Date` → datetime/conf 80; `Region` → geographic/conf 65; `Unit Price` → currency/conf 65. The resulting `schema_blueprint` is the contract that drives every decision in Agent 3.

6.3. Agent 3: Preprocessor

6.3.1. Overview and Purpose

Agent 3 (`agent3_preprocessor`) performs deterministic cleaning and feature construction, guided by the `schema_blueprint` from Agent 2. It turns raw values into an analysis-ready table. Unlike Agent 2, it uses no language model: every transformation is a fixed, logged rule, which makes the output reproducible and auditable.

6.3.2. Inputs and Outputs

- **Inputs:** the cached DataFrame `_df_cache`, the `schema_blueprint`, and `raw_profile`. The agent returns early if either the frame or the blueprint is missing.
- **Outputs:** `cleaned_df`, `cleaned_csv_path`, `scaling_params`, `preprocessing_log`, `data_quality`, a `column_ledger`, `category_normalization`, `rule_manifest`, and reliability metadata.

6.3.3. Profile Selection

Before cleaning, `_build_preprocessing_config` chooses a `strict`, `balanced`, or `lenient` configuration profile. A profile pinned by the caller is always honoured; otherwise `_detect_dataset_domain` inspects finance keywords and currency/percentage tags and selects `strict` for finance and sales data. Each profile fixes the currency limits, maximum plausible tax rate, reconciliation tolerances, and quality-score weights. Table 6-1 summarizes the profiles.

Table 6-1. Preprocessing profiles

Profile	currency _max_abs	max _tax_rate	recon _rel_tol	recon _abs_tol	quality weights (null / valfail / dup)
strict	1.1e9	0.30	0.01	0.50	0.55 / 0.35 / 0.10
balanced	1.0e9	0.40	0.02	1.0	0.50 / 0.40 / 0.10
lenient	1.0e10	0.60	0.05	2.0	0.40 / 0.30 / 0.30

6.3.4. Processing Workflow

Cleaning runs as an ordered, logged sequence. Each step records per-column null differences so its effect is visible. The stages, shown in Figure 6-6, are:

1. **Step 0: Deduplication** `dedup_exact_rows` removes exact duplicate rows, asserts the count matches Agent 1's profile, and aborts through `_assert_row_survival_or_abort` if fewer than half the rows survive (a safeguard against accidental data loss).

2. **Step 1: Currency cleaning** `_clean_currency_values` normalises monetary text and writes a `_parse_failed` flag for values it cannot parse.

3. **Step 2: Type coercion** `_coerce_types` casts each column to its intended type while tracking any new NaNs created by the cast.

4. **Step 3/3b: Text standardisation and re-dedup** text columns are canonicalised, then duplicates are removed again in case canonicalisation merged rows.

5. **Step 4: Imputation** `_impute` fills or drops missing values according to each column's policy. Supported strategies include mean, median, mode, `unknown_label`, forward fill, KNN imputer, iterative/multivariate imputer, `drop_rows`, `drop_column`, `flag_only`, and `none`. Currency and financial columns block imputation regardless of policy. Multivariate imputer results are cached across eligible columns.

6. **Step 5: Encoding** categorical columns are encoded for later analysis (one-hot when cardinality ≤ 10, top-8 + "Other" bucket otherwise, ordinal when order is present).

7. **Step 6: Outlier clipping** `_clip_outliers` caps extreme values using percentile bounds for skewed columns and the IQR $1.5\times$ rule otherwise.

8. **Step 7: Scaling** `_scale_columns` applies Min-Max scaling while preserving raw copies.

9. **Step 8: Date features** `_extract_date_features` derives calendar features from datetime columns.

10. **Step 9: Business metrics** `_derive_business_metrics` constructs profit, margin, and related fields where the necessary columns exist. Business metrics are then reconciled with ground-truth columns via `_reconcile_derived_metrics`.

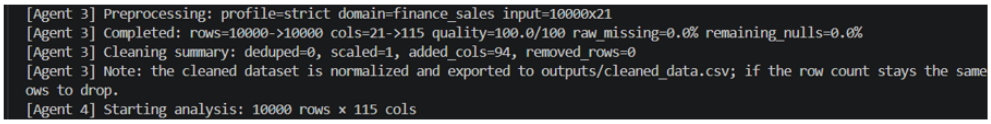

Figure 6-6. Workflow of Agent 3: Preprocessing Pipeline

6.3.5. Main Functions

`_normalize_currency_text`**: monetary parsing.** Financial exports can contain symbols, currency codes, parenthetical negatives such as `(123)`, and locale-specific separators (for example the European `1.234,56` versus the US `1,234.56`). This function converts those values to plain numeric strings so they can be cast to `float`. Figure 6-7 shows the separator logic: the function compares the last comma and last dot to decide which separator is decimal, then strips the thousands separator. Without this step, monetary columns would remain text and downstream financial metrics could not be computed.

```
last_comma = text.rfind(",")
last_dot = text.rfind(".")

if last_comma != -1 and last_dot != -1:
    decimal_sep = "," if last_comma > last_dot else "."
    thousands_sep = "." if decimal_sep == "," else ","
    text = text.replace(thousands_sep, "")
    text = text.replace(decimal_sep, ".")
elif last_comma != -1:
    tail = text[last_comma + 1:]
    if re.fullmatch(r"\d{2}", tail):         # 1234,56 ->
        1234.56
        text = text[:last_comma].replace(",", "") + "." + tail
    else:                                     # 1,234 -> 1234
        (thousands)
        text = text.replace(",", "")
```

Figure 6-7. Decimal/thousands-separator resolution in `_normalize_currency_text` (Agent 3)

`_build_canonical_category_map`**: fuzzy category normalisation.** This function merges similar categorical values using Levenshtein distance with guards: `FUZZY_MATCH_MIN_LENGTH=6`, `FUZZY_MATCH_MAX_DISTANCE=2`, first-char match, source $\leq 5\%$ rows and $\leq 1\%$ rare values, skip if ≤ 3 distinct values. It logs merges and flagged ambiguous values into `category_normalization`.

Business metrics and reconciliation. `_derive_business_metrics` uses whole-word matching to locate revenue, cost, unit, price, discount, and budget columns, then derives: profit $=$ rev $-$ cost, margin $\%$ $=$ profit/rev $\times$ 100, revenue per unit

$=$ rev/units, total revenue $=$ price $\times$ units, revenue after discount $=$ rev $-$ disc, discount % $=$ disc/rev $\times$ 100, budget variance $=$ rev $-$ budget, budget variance % $=$ var/budget $\times$ 100, total cost $=$ cost $+$ tax $+$ shipping, and days to ship $=$ (ship $-$ order).days. `_reconcile_derived_metrics` compares derived metrics against ground-truth columns using Pearson correlation and MAPE:

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\frac{|d_i - g_i|}{|g_i|}, \quad \text{diverged} \iff r < 0.99 \vee \text{MAPE} > 1.0\% \tag{6.2}$$

where d_i is the derived value and g_i is the ground-truth column value.

`_impute`: imputation strategies. Imputation strategies are summarized in Table 6-2.

Table 6-2. Imputation strategies supported by Agent 3

Strategy	Description	Applicable to
mean	Fill with column mean	Numeric (not currency)
median	Fill with column median	Numeric (not currency)
mode	Fill with column mode	Categorical
unknown_label	Fill with "Unknown"	Categorical
forward_fill	Forward-fill from previous row	Any (time-series)
KNN	K-Nearest Neighbors imputer	Numeric (not currency)
iterative	Multivariate IterativeImputer	Numeric (not currency)
drop_rows	Drop rows with missing values	Any (if missing rate low)
drop_column	Drop entire column	Any (if missing rate high)
flag_only	Create `_is_missing` flag, keep NaN	Any (e.g., currency)
none	No imputation	Any

Other functions. `_scale_columns` applies Min-Max normalisation,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{6.3}$$

while keeping `_raw` and `_scaled` copies and storing $x_{\min}$ and $x_{\max}$ in `scaling_params`, so the transform can be inverted later. `_derive_business_metrics` uses whole-word matching (`_find_col`, which avoids false hits such as "count" matching "Country") to locate revenue, cost, unit, price, discount, and budget columns. It then derives profit, margin, revenue per unit, discount percentage, and budget variance wherever the required columns exist, giving Agent 4 additional fields to summarise.

6.3.6. Validation and Data Quality

After transformation, `_validate_count_ranges` and `_validate_financial_constraints` record invalid counts and implausible tax rates in a `ColumnLedger`. `_compute_enhanced_quality_score` combines the remaining null rate, validation-failure rate, and duplication rate into a single 0 to 100 `data_quality` score. The ledger and score make the cleaning process reviewable: they show both the final data state and how much correction was needed. The quality score is:

$$\begin{aligned} Q &= w_c S_c + w_v S_v + w_d S_d, \\ S_c &= 100 - \text{missing\_penalty}\%, \quad S_v = 100 - \text{valfail}\%, \quad S_d = 100 - \text{dup}\% \end{aligned} \tag{6.4}$$

with weights per profile (Table 6-1), and a risk penalty of -5 for critical issues or -2.5 for high issues, clamped to $[0, 100]$.

6.3.7. Reliability and Pipeline Interaction

Every step appends to `preprocessing_log` with its null differences, and the cleaned frame is exported to `outputs/cleaned_data.csv`. Following the reliability model introduced in Equation 6.1, the agent combines the quality score with the row-survival ratio into a confidence value and marks the stage `ready` or `needs_review`. Critical failures trigger `_early_exit_with_error` so a corrupted result never propagates. On the sample dataset, Agent 3 parsed the currency fields into numeric values, extracted calendar features from the order and ship dates, one-hot encoded the region, item-type, and sales-channel columns, and derived business metrics, expanding the table from 14 to 58 columns with a data-quality score of 100.0/100. Because the sample contains no missing values, no imputation was required. The `cleaned_df` and `scaling_params` are the direct inputs to Agent 4.

6.4. Agent 4: Statistical Analysis

6.4.1. Overview and Purpose

Agent 4 (`agent4_analysis`) turns the cleaned dataset into descriptive statistics and chart artifacts. It produces summary statistics, correlations, trends, rankings, seasonality, anomalies, and regression fits, with saved figures for the charted results. Before running, it clears previous PNGs with `_clear_chart_dir` and writes each figure through `_save`

into `outputs/charts`, so each run starts from a clean output directory.

6.4.2. Inputs and Outputs

- **Inputs:** the `cleaned_df` and `scaling_params` from Agent 3 and the `schema_blueprint`; it errors out if no cleaned frame exists.
- **Outputs:** a `stats` dictionary holding all numerical results, a `chart_paths` list of saved figures, `chart_specs`, updated `errors`, and reliability metadata.

6.4.3. Processing Workflow

The agent runs a fixed battery of analyses over the selected columns, illustrated in Figure 6-8:

1. **Column selection** restricts analysis to meaningful fields, including `flag_leakage_columns` to exclude identifiers and leakage columns.
2. **Descriptive statistics** and a **correlation** matrix (Pearson and Spearman) summarise each column and the relationships between them.
3. **Growth rates** (MoM, QoQ), **rankings**, and **seasonality** describe how main measures change over time and across categories.
4. **Anomaly detection** (skew-aware z-score, log-z, IQR 3× far-out fence) and **regression trends** highlight unusual values and directional change.
5. **Distribution and derived-metric charts** visualise spread and the business fields created in Agent 3.
6. **Cross-dimensional analyses** (6 types) explore interactions like discount-return-rate, category-margin-trend, etc.
7. **Chart planning** uses signal-strength scoring (ANOVA η^2, Pareto, Cramér's V) to select and prioritize up to 10 charts.

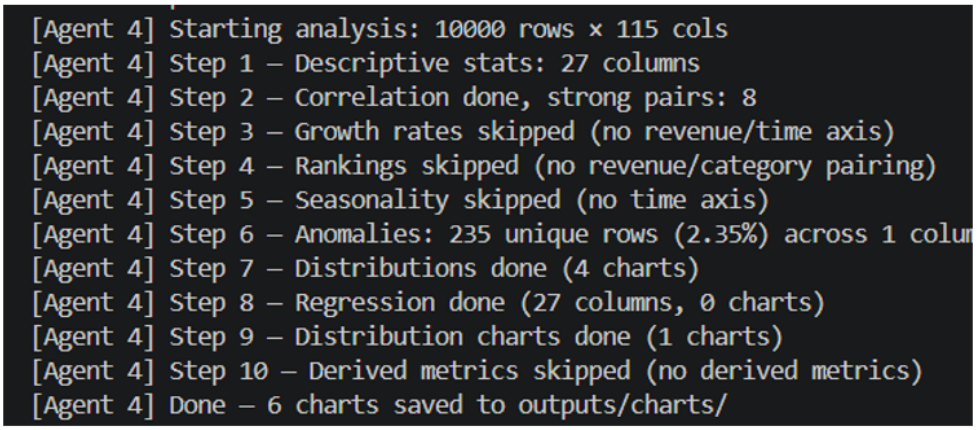

Figure 6-8. Workflow of Agent 4: Statistical Analysis

6.4.4. Main Functions

Column selection. `_numeric_cols` and `_categorical_cols` decide which columns are worth analysing, excluding validation columns (`_parse_failed`, `_range_failed`), scaling backups (`_raw`, `_scaled`), identifiers, datetime columns, leakage columns, and any column marked `analysis_allowed = false`. They run at the start of almost every other function, preventing meaningless output such as correlating an ID column. `flag_leakage_columns` detects leakage via name patterns (classifier, naive_bayes, _score, _proba, predicted_, id, index, clientnum, customerid, _uuid) and correlation signatures ($|r| > 0.98$ with exactly one column AND $|r| < 0.1$ with $\geq n-2$ others).

Summarisation. `_descriptive_stats` computes count, mean, median, standard deviation, variance, quartiles, skewness, and kurtosis for each numeric column. `_correlation` computes Pearson and Spearman matrices, renders a heatmap, and records the "strong" pairs ($|r| \geq 0.5$, labelled strong when $|r| \geq 0.7$), which gives the reporting stage the relationships most worth explaining.

`_detect_anomalies`**: statistical outlier flagging.** This function is skew-aware:

- If $|\text{skew}| \leq 1.0$: use z-score $z = \frac{x-\mu}{\sigma}$, flag $|z| > 3.5$.
- If $|\text{skew}| > 1.0$ and positive: use log-z on $\log(1+x)$.
- If heavy skew or negative values: use IQR with $3.0\times$ far-out fence: lower $= Q_1 - 3.0 \cdot \text{IQR}$, upper $= Q_3 + 3.0 \cdot \text{IQR}$.

It reports both per-column anomalies and the count of unique affected rows, which helps the pipeline estimate how noisy the dataset is. Columns with too few points or zero variance are skipped so the score stays well defined. Figure 6-9 shows the core z-score loop.

```
for col in _numeric_cols(df, schema_blueprint):
    s = df[col].dropna()
    if len(s) < 4:
        continue
    mean, std = s.mean(), s.std()
    if std == 0:
        continue
    z_scores = (df[col] - mean) / std
    anomaly_mask = z_scores.abs() > z_threshold
    anomaly_indices = df.index[anomaly_mask].tolist()
    if anomaly_indices:
        all_anomaly_indices.update(anomaly_indices)
```

Figure 6-9. z-score anomaly detection in `_detect_anomalies` (Agent 4)

Trend and regression analysis. `_growth_rates` uses a revenue column together with the `_month`/`_year` features from Agent 3 to plot month- and quarter-over-quarter change:

$$g_t = \frac{v_t - v_{t-1}}{v_{t-1}} \times 100 \tag{6.5}$$

`_top_bottom_rankings` ranks revenue across up to three categories, and `_seasonality` plots monthly averages with the best and worst months. `_regression_trends` fits an ordinary least-squares line on the `_month` axis with `scipy.stats.linregress`:

$$\hat{y} = \beta_1 x + \beta_0, \quad R^2 = r^2, \quad \text{significant iff } p < 0.05 \tag{6.6}$$

It records the slope, intercept, R^2, p-value, and whether the trend is significant ($p < 0.05$). Together, these functions convert the cleaned table into the directional findings a human analyst would usually inspect.

Chart planning and cross-dimensional analysis. `chart_planner.py` scores chart candidates using:

- ANOVA $\eta^2 = \frac{SS_{between}}{SS_{total}}$, skip if < 0.08.
- Pareto concentration $= 0.6 \cdot \text{top1\%} + 0.4 \cdot \text{top3\%}$, skip if < 30.
- Distribution: $|\text{skew}| + \text{outliers}$.
- Trend: $R^2 \cdot 100$.
- Seasonality: coefficient of variation.
- Scatter: $|r| \cdot 100$.
- Cramér's V: $V = \sqrt{\frac{\chi^2}{n \min(r-1, c-1)}}$.

Charts are deduplicated, sorted by priority, capped at 10, and grouped into sections (what_matters, shape, direction, relationships, watchlist). Cross-dimensional analyses include: discount-return-rate, category-margin-trend, rep-discount-margin, segment-order-value, region-shipping-cost, shipping-lead-time.

6.4.5. Reliability and Pipeline Interaction

The agent writes the `stats` dictionary and `chart_paths`, updates `errors`, and records reliability (0.9 when statistics were produced, otherwise 0.4) using the same `update_reliability` mechanism described for Agent 1. On the sample dataset, Agent 4 produced descriptive statistics for 22 numeric columns, a correlation heatmap that revealed a very strong positive relationship between Unit Price and Unit Cost ($r = 0.99$), and regional and item-type revenue rankings, seasonality, and derived-metric

charts (16 charts in total). These statistics and charts form the evidence base that the validation and reporting stages consume.

6.5. Agent 5: Output Validation & Trust Gate

6.5.1. Overview and Purpose

Agent 5 (`agent5_output_validator`) is the quality gate before report generation. It performs deterministic contract checks and a Cohen's kappa agreement test between the LLM's semantic tags and heuristic type sniffing, giving the system a ground-truth-free measure of tag reliability.

6.5.2. Inputs and Outputs

- **Inputs:** `cleaned_df`, `schema_blueprint`, `stats`, `chart_paths`, `raw_profile`, `preprocessing_log`, `category_normalization`, `column_ledger`, `rule_manifest`.
- **Outputs:** `validation_report` with `passed` (bool), `score`, `checks` (pass/warn/-fail per check), `flagged_issues`, `cohen_kappa`, and updated `reliability`.

6.5.3. Processing Workflow

The agent proceeds in two tiers:

Tier 1: Deterministic Contract Checks

These checks are hard rules with no LLM involvement:

- **Row reconciliation:** `row_accounting` counts (raw rows, cleaned rows, dropped rows) must match.
- **Schema consistency:** `cleaned_df` columns must align with `schema_blueprint`.
- **Quality score:** `data_quality` must be in $[0, 100]$.
- **Stats numeric sanity:** Descriptive statistics must be finite; correlations in $[-1, 1]$.
- **Chart artifact integrity:** Chart files must exist and be non-empty.
- **Business-rule validation:** Worst failure $\leq$ `MAX_VALIDATION_FAIL_PCT` = 15%.
- **Category normalization safety:** Reject merges changing $> 5\%$ rows or merging two frequent values.
- **Trend sample sufficiency:** Significant trends need $n \geq$ `MIN_TREND_SAMPLE_SIZE` = 10.

Tier 2: Cohen's Kappa

`_cohen_kappa_score` computes agreement between Agent 2's LLM `intended_type` and the heuristic sniffer, coarsened to {numeric, datetime, boolean, string}. Cohen's kappa is:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad p_o = \text{observed agreement}, \quad p_e = \sum_k \hat{p}_k^A \hat{p}_k^B \tag{6.7}$$

where $\hat{p}_k^A$ and $\hat{p}_k^B$ are the proportions of each category assigned by raters A and B. The threshold is `MIN_ACCEPTABLE_KAPPA` = 0.4 ("fair", Landis & Koch [18]).

6.5.4. Scoring and Gating

The overall validation score is:

$$\text{VS} = 100 \cdot \frac{\text{passed} + 0.5 \cdot \text{warned}}{\text{total}} \tag{6.8}$$

The pipeline passes (`passed = True`) iff `failed_checks == 0`. A failed validation stops the pipeline before Agent 6, so unchecked outputs do not reach the report.

```
[Agent 5] Starting output validation
[Agent 5] Completed: score=100.0/100 passed=9/9 warned=0 failed=0 kappa=1.0
```

Figure 6-10. Agent 5 output validation implementation: Tier-1 contract checks and Cohen's kappa gate

6.6. Agent 6: Insight Report Generator

6.6.1. Overview and Purpose

Agent 6 (`agent6_insight_report_generator`) turns validated statistics, charts, and facts into an HTML/PDF report. It combines deterministic fact extraction with LLM-generated narrative, claim grounding, and a hybrid composition step that prioritizes verified content.

6.6.2. Inputs and Outputs

- **Inputs:** `cleaned_df`, `schema_blueprint`, `stats`, `chart_paths`, `chart_specs`, `validation_report`, `preprocessing_log`, `category_normalization`, `reliability`, `run_id`.
- **Outputs:** `insight_facts` (deterministic facts), `insight_narrative` (hybrid narrative), `report_path` (PDF/HTML), updated `reliability`.

6.6.3. Processing Workflow

The agent proceeds in seven stages:

1. Deterministic Fact Extraction

`_extract_insight_facts` extracts verifiable facts from the state without using the LLM:

- Dataset shape and shape-change transparency (why cleaned cols/rows differ: one-hot/ordinal/derived/date-feature/audit-trail/validation-flag).
- Quality facts (data_quality, missing rates, validation score).
- Per-column missing-value actions reconciled against reality.
- Top correlations (excluding leakage/formulaic).
- Growth rates, rankings, profit breakdown, cross-dimensional findings.
- Category normalisation merges and flagged values.
- Anomalies (per-column and unique rows).
- Significant trends (OLS, R^2, p-value).
- Validation facts (Cohen's kappa, checks).
- Reliability (stage confidences, overall, decision_readiness).
- Chart summaries.

2. LLM Narrative Generation

`_call_llm_for_narrative` calls Groq (`qwen/qwen3.6-27b`) with a strict system prompt that forbids hallucination and requires using only provided facts. It returns a JSON with: `executive_summary`, `key_findings`, `story` (what_happened/why/next), `chart_captions`, `glossary_terms`, `plain_language_insights`, `bottom_line`, `risks_and_caveats`, `recommendations`. Prompt facts are compacted to stay under `MAX_NARRATIVE_PROMPT_CHARS=7000`. If Groq fails, the system falls back to Gemini (with the same failover chain) and finally to a deterministic fallback.

3. Claim Grounding

`_check_narrative_grounding` verifies that every number in the narrative matches a computed fact within tolerance:

$$\text{grounded} \iff |c-k| \leq \max(1.0, 0.05 \cdot |k|) \tag{6.9}$$

for some known fact k. The grounding confidence is:

$$\text{conf} = \frac{\text{grounded}}{\text{checked}} \tag{6.10}$$

and the overall report confidence is 0.95 (LLM narrative) or 0.55 (fallback), multiplied by $(0.7 + 0.3 \cdot \text{grounding})$.

4. Hybrid Composition

`_compose_hybrid_narrative` always renders the deterministic floor, then layers the LLM output only where it passes hygiene (valid story, no jargon via `_JARGON_PATTERNS`, known chart ids). `_lint_plain_language` swaps residual jargon back to deterministic wording.

5. Recommendation Grounding

`_ground_recommendations` checks that each recommendation is anchored to a reported finding and traceable money figures.

6. Table-Cell and Raw-Column Guards

`_validate_no_empty_required_cells` and `_validate_raw_column_count` fail loudly on empty required cells or shape drift.

7. Rendering

The report is rendered with Jinja2 (`insight_report.html.jinja`) using base64-embedded PNGs and embedded ECharts option JSON for interactive charts. KPI cards and narrative sections are grouped in the template. `WeasyPrint` converts HTML to PDF, with an HTML fallback if PDF libraries are unavailable. Output is saved to `outputs/reports/<run_id>/insight_report.pdf|html`.

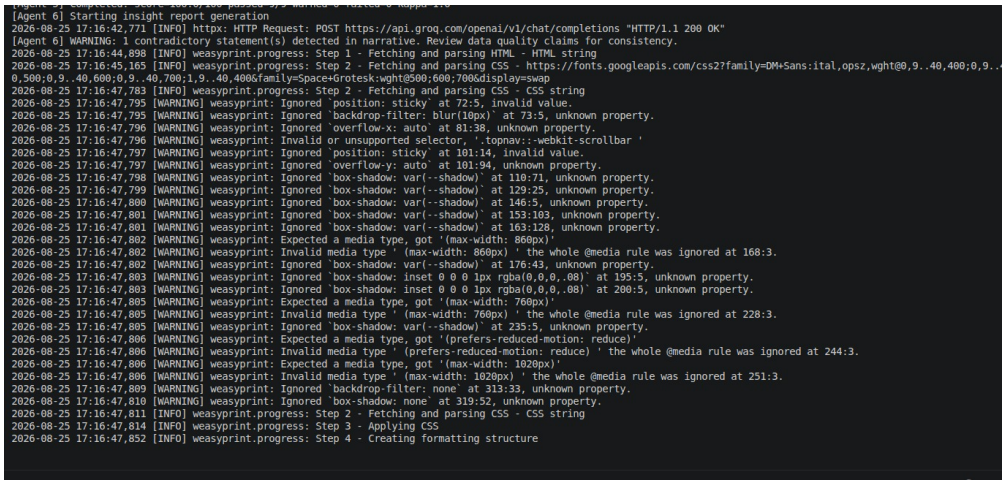

Figure 6-11. Agent 6 insight report generation implementation: facts → LLM narrative → hybrid+grounding → Jinja2/WeasyPrint

6.7. Backend API Implementation

6.7.1. App and Configuration

The FastAPI app is created via `create_app()`, which registers routers for health, auth, analysis, reports, jobs, and chat. CORS is enabled via `middleware/cors.py`. Configuration is managed by `Settings` (via `@lru_cache`) with environment variables for uploads directory, outputs, max upload size (50 MB), allowed extensions ('.csv'), SSE poll interval (0.15s), keepalive (15s), max concurrent jobs (4), job TTL (3600s), PostgreSQL connection, RAG embedding model (`BAAI/bge-base-en-v1.5`), and Hugging Face token.

6.7.2. Endpoints

Table 5-1 summarizes the main endpoints. Authentication uses JWT-style tokens with PBKDF2-SHA256 password hashing (via `passlib`). User and job data are stored in PostgreSQL tables (`app_users`, `analysis_jobs`) with a file-based fallback (`outputs/analysis_jobs.json`) if PostgreSQL is unavailable.

6.7.3. Job Manager and Pipeline Runner

The `JobManager` maintains thread-safe `Job` dataclasses with status (queued/processing/-completed/failed), per-agent progress, append-only events for SSE, state, result, chat history, and RAG status. Jobs are persisted through `PostgresJobStore` or `FileJobStore`. The `PipelineRunner` runs the compiled LangGraph in a daemon thread, streams values, and emits SSE events when agents complete and milestones are reached.

6.7.4. SSE Streaming

Server-Sent Events are implemented in `sse.py` with format: `event: <name>\n data: <json>\n\n`. Past events are replayed on connection, polling occurs every 0.15s, keepalive is sent every 15s, and the stream closes on

terminal events. The stream is multi-subscriber safe.

6.7.5. RAG Dataset Chat

The RAG service uses `BAAI/bge-base-en-v1.5` embeddings via the Hugging Face Inference API when the required `DATABASE_URL` and Hugging Face token are configured. pgvector stores embeddings for row documents from the original uploaded CSV and fact documents derived from the generated analysis context. During retrieval, the service fetches top row documents (default $K = 8$), top column/statistic fact documents (default $K = 6$), and always includes dataset-wide facts such as summaries, validation results, correlations, rankings, anomalies, and existing charts. The chat service passes the retrieved context and recent chat history to Groq $\rightarrow$ Gemini $\rightarrow$ deterministic fallback for response generation. If the database or embedding service is unavailable, the fallback answers from deterministic summary facts rather than claiming vector retrieval. It can also generate a fresh chart based on the query.

6.8. Frontend Implementation

6.8.1. Stack and Structure

The frontend (`frontend/AnalyzeAI/`) uses React 19.2.7, react-router-dom 7.18.1, TailwindCSS 4.3.2, local shadcn-style UI primitives built with class-variance-authority, framer-motion, lucide-react, and Vite 8.1.0. No Redux is used; state is managed via React Context (`AuthContext`, `ThemeContext`) and per-page useState. Charts are rendered from backend ECharts JSON; there is no npm charting library.

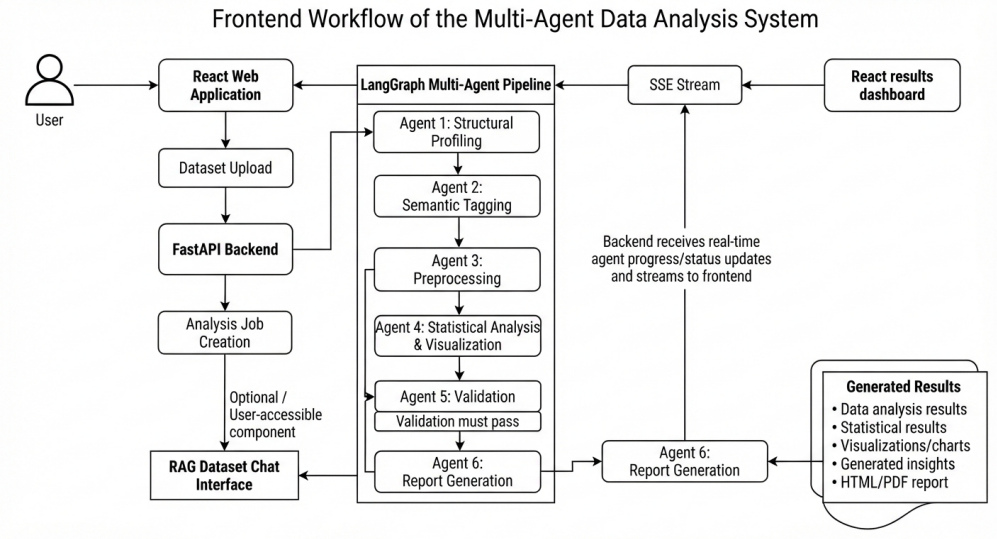

Figure 6-12. Frontend workflow: user upload, SSE progress, results, and RAG chat

6.8.2. Routing and Authentication

BrowserRouter provides routes: `/` (LandingPage), `/analyze` + `/analyze/:jobId` (AnalyzePage, wrapped in RequireAuth), `/history` (HistoryPage), `/login`, `/signup`,

/profile. RequireAuth redirects to /login if no user is in localStorage.

6.8.3. API Client and SSE

The API client (src/lib/api.js) provides analyzeCsv (FormData upload), subscribeToJobStream (EventSource listening for all event types), fetchJobResult, fetchJobs, chat, and reportDownloadUrl. The EventSource listens for csv_loaded, pipeline_started, agent_progress, charts_generated, validation_passed/failed, report_generated, pipeline_finished/completed/failed, and error.

6.8.4. Main Components

- **AnalyzePage:** Handles file upload, displays live agent progress with color coding (blue for Agent 1, cyan, emerald, amber, rose, violet), shows results dashboard with KPI cards, and includes DatasetChat for grounded Q&A. - **HistoryPage:** Lists past analyses with rows, cols, quality, confidence, duration. - **AppNavbar, ThemeToggle, and local reusable UI primitives.**

7. RESULTS AND ANALYSIS

The results below come from a complete pipeline run on the `10000 Sales Records.csv` dataset. The outputs include per-agent logs, diagnostic JSON, generated charts, and the final HTML/PDF report. All reported results come from executions of the implemented codebase.

7.1. Pipeline Execution Summary

A complete run on the sample dataset produces the following per-stage summary (captured from the run diagnostics):

- **Agent 1:** Profiled 10000 rows $\times$ 14 columns; missing 0.0%, duplicates 0.
- **Agent 2:** Type sniffing {string:5, datetime:2, numeric:7}; blueprint built for 14 columns.
- **Agent 3:** profile=strict, domain=finance_sales, input 10000$\times$14 $\rightarrow$ cleaned 10000$\times$58, quality=100.0/100, remaining nulls 0.0%.
- **Agent 4:** 22 descriptive columns, 10 strong correlation pairs, 52 anomalous rows (0.52%), 16 charts saved.
- **Agent 5:** validation score 100/100, 9/9 checks passed, Cohen's kappa = 1.0.
- **Agent 6:** narrative source = Groq, 21/21 claims grounded, report written to `outputs/reports/<run_id>/insight_report.html` (HTML fallback; see below).

7.1.1. Comparison with a General-Purpose Qwen LLM

The same automated insight-generation task was also evaluated against a general-purpose Qwen LLM using the `10000_Sales_Records.csv` dataset. The comparison shows the practical value of separating deterministic analysis from language generation. The multi-agent pipeline completed in approximately 52 seconds, while the standalone Qwen run took approximately 4 minutes 12 seconds, making it about 4.8 times slower.

Table 7-1. Comparison of the multi-agent pipeline and a general-purpose Qwen LLM

Dimension	Multi-agent data analyst	General-purpose Qwen LLM
Runtime	Approximately 52 seconds	Approximately 4 minutes 12 seconds, or 4.8 times slower
Numerical accuracy	Exact against the source data; totals, regional and category breakdowns, and rankings matched the ground truth	Rankings and percentages were directionally correct, but absolute dollar figures had a consistent 10 times scaling error. For example, total revenue was reported as $1.33B instead of the actual $13.33B
Self-validation	Output Validation cross-checked 23 of 23 narrative figures against pipeline output, with confidence 1.0, and a separate ground-truth row-count reconciliation was performed	No validation layer was used, so numerical errors could reach the final output without being detected
Statistical depth	Pearson correlations with tautology filtering, adaptive z-score/IQR outlier detection, quarterly and year-over-year trends, and shipping-time breakdowns across dimensions	Executive-summary totals, top categories and regions, and high-level recommendations; no outlier detection or correlation analysis
Data quality	Duplicate and missing-value handling, anomaly flagging with estimated business impact of $412K across 52 rows, and a reported 100/100 validation pass	No data-quality or anomaly-detection section
Traceability	Figures and charts were sourced from deterministic pipeline computations and can be reproduced	The output did not show how figures were derived, so it was not reproducible from the summary alone
Output structure	A 27-page report covering the overview, findings, correlations, distributions, anomalies, recommendations, technical appendix, and glossary	A single-pass Markdown summary of approximately five sections, without an appendix or methodology trail
Actionability	Findings were tied to specific evidence, such as Snacks aver-	Recommendations were broader and less tied to

The multi-agent pipeline was therefore approximately five times faster and was the only system in this comparison whose output was independently verifiable against the source data. The general-purpose LLM produced directionally useful results, but its consistent 10 times scaling error shows why a validation layer matters. Output Validation is not only an additional reporting step; it prevents numerical errors of this kind from reaching the user unnoticed.

7.2. Per-Agent Outputs

7.2.1. Agent 1: Structural Profiling

The agent loaded the CSV file, detected the comma delimiter, and profiled all 14 columns. No missing values or duplicate rows were detected in this sample. The raw profile became the baseline for all downstream stages.

7.2.2. Agent 2: Semantic Tagging

Agent 2 produced a 14-column schema blueprint (the LLM output is used when available, with a heuristic fallback). Representative tags from the sample run:

- `Order ID` → identifier / conf 80
- `Order Date`, `Ship Date` → datetime / conf 80
- `Region`, `Country` → geographic / conf 65
- `Item Type`, `Sales Channel`, `Order Priority` → categorical label / conf 75
- `Units Sold` → count / conf 65
- `Unit Price`, `Unit Cost`, `Total Revenue`, `Total Cost`, `Total Profit` → currency / conf 65

On this run the heuristic type sniffer and the assigned intended types agreed on every column, yielding a Cohen's kappa of 1.0 (perfect agreement), later confirmed by Agent 5.

7.2.3. Agent 3: Preprocessing

Using the automatically selected `strict` profile (finance_sales domain), the agent performed:

- Currency cleaning on the five monetary columns (Unit Price, Unit Cost, Total Revenue, Total Cost, Total Profit).
- Type coercion of object columns to their intended types, with `format="mixed"` datetime parsing on Order Date and Ship Date.
- Imputation: no missing values were present, so no imputation was required.

- Adaptive outlier clipping on eligible numeric columns (currency/identifier columns are excluded).
- Min-Max scaling of eligible numeric columns, preserving `_raw` and `_scaled` copies.
- Date-feature extraction and one-hot encoding of Region, Item Type, and Sales Channel; ordinal encoding of Order Priority.
- Business-metric derivation with ground-truth reconciliation against the existing Total Profit column.

The cleaned dataset expanded from 14 to 58 columns due to one-hot encoding and derived features, with a data-quality score of 100.0/100 and no flagged validation issues.

7.2.4. Agent 4: Statistical Analysis and Visualization

Agent 4 produced descriptive statistics, correlations, trends, and 16 charts. Table 7-2 lists a representative subset. The sample run produced these findings:

- **Descriptive stats:** 22 numeric columns were summarised (count, mean, median, standard deviation, variance, quartiles, skewness, kurtosis).
- **Correlation:** 10 pairs with $|r| \geq 0.5$ were identified. The strongest was `Unit Price` vs `Unit Cost` ($r = 0.99$), followed by `Unit Price` vs `Total Cost` ($r = 0.75$) and `Unit Price` vs `Total Revenue` ($r = 0.73$); `Units Sold` correlated moderately with `Total Profit` ($r = 0.60$) and `Total Revenue` ($r = 0.52$).
- **Anomalies:** 52 rows (0.52%) were flagged as statistical outliers using the skew-aware detector.
- **Trends:** OLS regression was fitted on the available time axis; none of the 8 candidate trends met the significance and minimum-sample thresholds on this sample, so no trend was reported as significant.
- **Rankings and seasonality:** Revenue and profit were ranked across Region and Item Type, and monthly/quarterly revenue and seasonality were charted.

Table 7-2. Representative charts generated by Agent 4 (16 total)

Chart Name	Type	Description
correlation_heatmap.png	Heatmap	Pearson correlations among numeric columns
boxplot_numeric_cols.png	Boxplot	Spread of numeric columns
total_revenue_histogram.png	Histogram	Distribution of total revenue
scatter_unit_price_vs_unit_cost.png	Scatter	Unit price vs unit cost
monthly_total_revenue_growth.png	Line	Month-over-month revenue
monthly_total_revenue_seasonality.png	Line	Monthly revenue seasonality
top_5_region_total_revenue.png	Bar	Top regions by revenue
top_5_item_type_total_revenue.png	Bar	Top item types by revenue
top_5_region_total_profit.png	Bar	Top regions by profit
shipping_lead_time_distribution.png	Histogram	Shipping lead-time distribution

7.2.5. Agent 5: Validation

All nine Tier-1 deterministic contract checks passed on the sample run:

- Row reconciliation: raw 10000, cleaned 10000, dropped 0.
- Schema consistency: all 58 cleaned columns matched the blueprint.
- Quality score: 100.0 within $[0, 100]$.
- Stats numeric sanity: all descriptive values finite; correlations in $[-1, 1]$.
- Chart artifact integrity: all 16 chart files present and non-empty.
- Business-rule validation: worst failure $0\% \leq 15\%$.
- Category normalization safety: no unsafe merges.
- Trend sample sufficiency: no significant trend fell below the minimum sample size.

Cohen's kappa between the assigned intended types and the heuristic sniffer was 1.0 (perfect agreement). The overall validation score was 100/100 (9/9 checks passed), so the gate passed and Agent 6 proceeded.

Table 7-3. Cohen's kappa validation result for semantic tagging

Field	Sample run value
Compared decisions	Agent 2 assigned intended type vs. heuristic type sniffer
Compared columns	14
Cohen's kappa	1.0
Disagreements	0
Acceptance threshold	`MIN_ACCEPTABLE_KAPPA` = 0.4
Interpretation in this project	Semantic typing passed the agreement gate; this is an internal consistency check, not an external human-label accuracy score

7.2.6. Agent 6: Report Generation

The agent extracted deterministic facts, generated a narrative with Groq (Qwen3-27B), and grounded every numeric claim: 21 of 21 checked claims matched the computed facts (grounding confidence 1.0). The report includes KPI cards, charts, a plain-language executive summary, findings, a three-part story (what happened / why / next), recommendations, risks, and a glossary. On the test machine, WeasyPrint's native libraries were not installed, so the report was written as a self-contained HTML file (`insight_report.html`); the same code produces a PDF when those libraries are present.

7.3. Performance and Reliability

Table 7-4 shows the per-stage confidence scores and overall decision readiness recorded by the reliability mechanism on the sample run.

Table 7-4. Reliability and confidence propagation (sample run)

Stage	Confidence	Evidence	Readiness
Agent 1	1.00	No missing values or duplicates	ready
Agent 2	*n/a*	LLM-to-heuristic agreement $\kappa = 1.0$	ready
Agent 3	1.00	Quality = 100.0/100	ready
Agent 4	0.90	Statistics produced, 16 charts	ready
Agent 5	1.00	9/9 checks passed	ready
Agent 6	0.95	Grounding 21/21, narrative = Groq	ready
Overall	0.97	Mean of the five stages reporting a numeric confidence; decision_readiness = ready (≥ 0.85)	

7.3.1. API Usage and Runtime Observability

The system uses external APIs in bounded locations: Agent 2 calls Groq for schema tagging with Gemini fallback; Agent 6 calls Groq for narrative generation with Gemini fallback; and the RAG chat uses Hugging Face embeddings plus Groq/Gemini/-fallback answer generation when vector retrieval is configured. The code records provider/model/key-fingerprint usage for Agent 2 through the API-key usage tracker, and the persisted SSE event log records start/completion timestamps for each pipeline agent. The current implementation does not persist prompt tokens, completion tokens, total tokens, or per-call API response latency. For that reason, token counts and API latency values are not reported as numerical results in this project report.

Table 7-5 reports only execution times that can be derived from the saved API job events in `outputs/analysis_jobs.json`. These are pipeline-stage wall-clock durations, not provider-level API latency measurements.

Table 7-5. Pipeline runtime derived from persisted API event timestamps

Stage	Duration (seconds)	Source
Agent 1	0.38	SSE running/completed timestamps
Agent 2	4.66	SSE running/completed timestamps
Agent 3	1.22	SSE running/completed timestamps
Agent 4	6.72	SSE running/completed timestamps
Agent 5	0.01	SSE running/completed timestamps
Agent 6	2.48	SSE running/completed timestamps
Total pipeline	24.07	`pipeline_started` to `pipeline_finished`

Table 7-6. API and LLM metric availability in the current implementation

Metric	Status	Evidence / required change
LLM/API models used	Available	Model constants and service configuration identify Groq, Gemini, and Hugging Face embedding models
Number of LLM calls	Partially available	Agent 2 captures provider use; Agent 6 and chat calls are not yet persisted as a complete call count
Input / prompt tokens	Not recorded	Requires reading provider usage metadata after each response and storing it in the run state
Output / completion tokens	Not recorded	Requires provider response usage capture
Total tokens	Not recorded	Requires prompt and completion token capture per call
API response latency	Not recorded	Requires timing wrapper around Groq, Gemini, and Hugging Face calls
Agent execution time	Derivable	Persisted SSE timestamps record agent running/completed events
Total pipeline time	Derivable	Persisted `pipeline_started` and `pipeline_finished` events
Fallback calls	Partially observable	Fallback paths exist in code; exact fallback-attempt counts are not yet persisted

Future runs can measure token usage and API latency by adding a shared API-call metrics wrapper around Groq, Gemini, and Hugging Face calls. That wrapper should record provider, model, agent, purpose, success/failure, fallback attempt, latency in milliseconds, and any available `prompt_tokens`, `completion_tokens`, and `total_tokens`. The metrics should then be stored in the shared state and included in the diagnostics JSON. Until that instrumentation exists and the pipeline is rerun, this report should not claim numerical token counts or provider response latencies.

7.3.2. RAG Chat Evaluation Status

The dataset chat works as a retrieval-augmented system when PostgreSQL/pgvector and the Hugging Face token are configured. The retrieval corpus contains two document

groups: row documents from the original uploaded CSV and fact documents from the deterministic analysis result. The retriever embeds documents and user queries with `BAAI/bge-base-en-v1.5`. Row documents are ranked by vector distance, while fact types such as summaries, validation, correlations, rankings, anomalies, and chart references are always included so broad analytical questions stay grounded in computed results. The retrieved context and recent chat history are then passed to the LLM, and the final answer is generated through Groq, Gemini fallback, or deterministic fallback.

The implementation has functional tests for RAG triggering, context construction, retrieval branches, fallback behavior, and validated on-demand chart generation. It does not yet include a labelled retrieval benchmark, so quantitative retrieval metrics are not reported as achieved results. Table 7-7 separates implemented behavior from evaluation work that still needs a labelled experiment.

Table 7-7. RAG chat implementation and evaluation status

Item	Status	Notes
RAG implementation	Implemented	Uses original CSV row-docs plus generated analysis fact-docs
Embedding model	Implemented	Hugging Face `BAAI/bge-base-en-v1.5`
Vector store	Implemented when configured	PostgreSQL with pgvector; fallback summary chat is used if unavailable
Retrieved rows	Configured	Default top-K rows = 8
Retrieved fact docs	Configured	Default top-K column/stat facts = 6, plus always-included dataset-wide facts
Answer grounding	Implemented structurally	The prompt restricts answers to retrieved facts/rows; fallback uses deterministic summary facts
Precision@K	Requires additional evaluation	Needs labelled relevant/not-relevant documents per test question
MRR	Requires additional evaluation	Needs the rank of the first relevant retrieved document per question
nDCG@K	Requires additional evaluation	Needs graded relevance labels, e.g. 0/1/2 relevance levels
Recall@K	Conditionally applicable	Valid only if the complete relevant document set is labelled over the indexed corpus

For a future quantitative RAG evaluation, a test set should be created with representative user questions, the expected relevant row/fact documents for each question, and optional graded relevance labels. Precision@K should report how many retrieved documents are relevant, MRR should report how early the first relevant document appears, and nDCG@K should reward highly relevant documents appearing near the top. Recall@K should only be used when all relevant documents are known for each query; otherwise it would overstate what can be measured from the current sampled row index. No Precision@K, Recall@K, MRR, or nDCG@K values are claimed in this report because that labelled experiment has not yet been performed.

7.4. Error Analysis and Validation Discussion

The sample run showed the following error-handling and validation behavior:

- **Currency parsing:** The European vs US decimal resolution (Figure 6-7) correctly parsed the monetary columns to numeric values, including accounting negatives and mixed symbols.
- **Datetime coercion:** The `format="mixed"` flag in `pd.to_datetime` avoids a class of bug in which mixed date formats are silently converted to NaT; the order and ship dates were parsed without row loss.
- **Outlier clipping:** Adaptive IQR/percentile clipping caps extreme values while excluding currency and identifier columns, preventing distortion of the statistics without discarding genuine business outliers.
- **Ground-truth reconciliation:** The reconciliation guardrail ($r \geq 0.99$, MAPE $\leq$ 1%) cross-checks each derived metric against any equivalent source column; on this sample no divergence was flagged.
- **Cohen's kappa:** The $\kappa = 1.0$ agreement between the assigned types and the heuristic sniffer provides evidence that the semantic tagging is reliable without requiring ground-truth labels, giving the system a trust signal without external labels.
- **Claim grounding:** All 21 numeric claims in the generated narrative were verified against the computed facts within tolerance.

These results show that the multi-agent pipeline processes a raw CSV file through all six stages and produces a validated, grounded analytical report with minimal human intervention.

8. FUTURE WORK

The current system runs the full workflow from upload to report generation. Future work includes:

- **API-level multi-format ingestion:** Extend the FastAPI upload route beyond CSV so the web interface can accept the Excel, JSON/JSONL, and Parquet formats already supported by the internal Agent 1 loader. Direct database connections remain a future extension.
- **Model-agnostic provider configuration:** Allow users to configure their preferred LLM providers (Groq, Gemini, OpenAI, Anthropic, local models via Ollama) through a settings panel.
- **More chart families:** Add Sankey diagrams, treemaps, geographic maps, and time-series decomposition plots.
- **Auth hardening:** Implement proper OAuth2/OpenID Connect with refresh tokens, role-based access control (RBAC), and multi-tenancy for organizational use.
- **Larger-scale evaluation:** Conduct systematic benchmarking on diverse datasets (100+ CSVs) to quantify accuracy, speed, and reliability across different domains and data quality levels.
- **Interactive report customization:** Allow users to select chart types, add custom metrics, and modify the narrative tone through a visual editor.
- **Continuous learning:** Implement feedback loops where user corrections (e.g., fixing a mis-tagged column) are stored and used to fine-tune the semantic tagging and preprocessing rules.

These enhancements would make the system easier to adapt to different datasets, deployments, and user needs.

9. CONCLUSION

This project developed a multi-agent system for automated data analysis and insight generation from financial and sales CSV data. The system has six specialized agents: structural profiling, semantic tagging, context-aware preprocessing, statistical analysis and visualization, output validation, and hybrid report generation. LangGraph orchestrates these agents with conditional routing and a hard validation gate. A FastAPI backend provides file uploads, job management, Server-Sent Events for live progress streaming, and grounded RAG dataset chat. A React 19 frontend lets users upload datasets, monitor pipeline execution, view interactive charts, and download HTML/PDF reports.

The system separates responsibilities across agents and keeps an audit trail through logged transformations and confidence propagation. Reliability is supported by deterministic contract checks, Cohen's kappa agreement scoring, and claim grounding before final report generation. On the representative sample dataset, the pipeline processes a raw CSV file from ingestion to a formatted report (HTML, and PDF when WeasyPrint's native libraries are available) with KPI summaries, charts, plain-language findings, and actionable recommendations.

The project achieves its primary objectives: (1) an autonomous multi-agent system that converts raw data into professional reports without manual coding, and (2) a web interface that makes financial analytics more accessible to small and medium businesses. The implementation is usable as a working prototype and has clear areas for future extension.

The main contribution is a practical framework for agentic data analysis that combines deterministic algorithms with controlled LLM reasoning. It is designed for settings such as education, non-profits, and small businesses where data is available but technical analytics expertise is limited.

10. APPENDICES

Appendix A: Dataset Details

The sample dataset used for validation and demonstration is `10000 Sales Records.csv`, available in the project repository. It contains 10,000 rows and 14 columns, representing simulated global sales transactions with the following columns:

- `Region`: Geographic region (categorical, e.g. Sub-Saharan Africa, Europe, Asia).
- `Country`: Country of the transaction (categorical).
- `Item Type`: Product category (categorical, 12 distinct values).
- `Sales Channel`: Online or Offline (categorical).
- `Order Priority`: Order priority code (categorical: H, M, L, C).
- `Order Date`, `Ship Date`: Order and shipment dates (datetime).
- `Order ID`: Unique transaction identifier (integer).
- `Units Sold`: Quantity sold (integer count).
- `Unit Price`, `Unit Cost`: Per-unit price and cost (currency).
- `Total Revenue`, `Total Cost`, `Total Profit`: Aggregate monetary fields (currency).

The sample is clean (no missing values, no duplicates) and exercises the full pipeline: identifiers, dates, categories, and multiple monetary columns that support derived-metric reconciliation. The system itself is not tied to this dataset and accepts arbitrary tabular CSV files.

Appendix B: Cost Estimate

Table 10-1. Indicative project cost estimate

Item	Estimate	Notes
Development laptop and local machine	Existing	Already available for development
Python, FastAPI, LangGraph, plotting stack	Existing	Open-source software stack
React, Vite, TailwindCSS	Existing	Open-source frontend stack
PostgreSQL (optional)	Existing/FOSS	Free for local/-cloud use
LLM API usage (Groq & Gemini)	Variable	Usage-based; small per-run cost
Internet and miscellaneous expenses	Variable	Project maintenance cost

The cost estimate is lightweight because the project uses open-source libraries and an existing development environment. The main variable cost comes from API usage during testing and report generation. That cost can be managed by keeping the heuristic fallback path available, using the schema cache, and avoiding unnecessary repeated model calls. Agent 2 sends only column metadata and a few sample values rather than full rows, which limits prompt size by design; exact token counts are not currently persisted by the implementation.

Appendix C: Project Timeline

Table 10-2. Project timeline summary

Task	Status	Phase
Requirement analysis and proposal	Complete	Initial
Agent 1 to 4 implementation and integration	Complete	Development
Agent 5 to 6 implementation	Complete	Development
Backend API and frontend development	Complete	Development
Testing, evaluation, and debugging	Complete	Finalization
Documentation and report submission	Complete	Final

All major tasks are complete. The project moved from proposal to a working system with a web interface, API, and six-agent pipeline. The final phase focused on integration testing, report generation, and documentation.

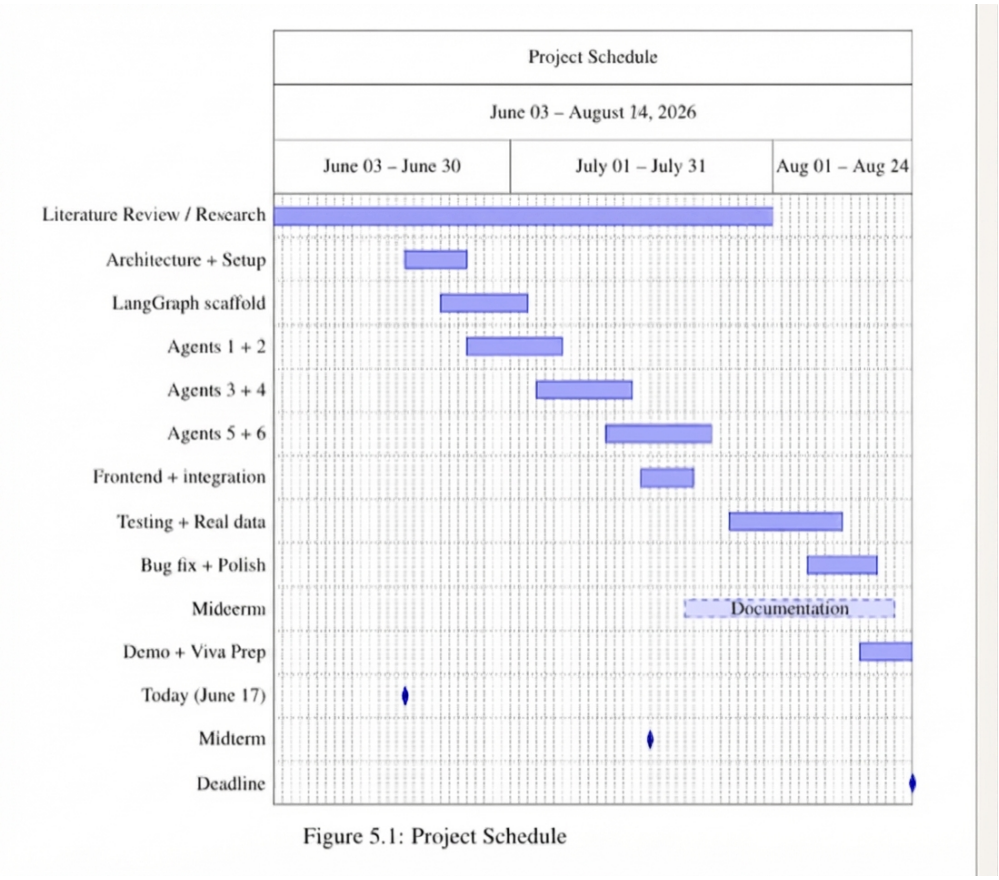

Figure 5.1: Project Schedule

Figure 10-1. Project schedule and milestones

Appendix D: Sample Generated Report Screenshots

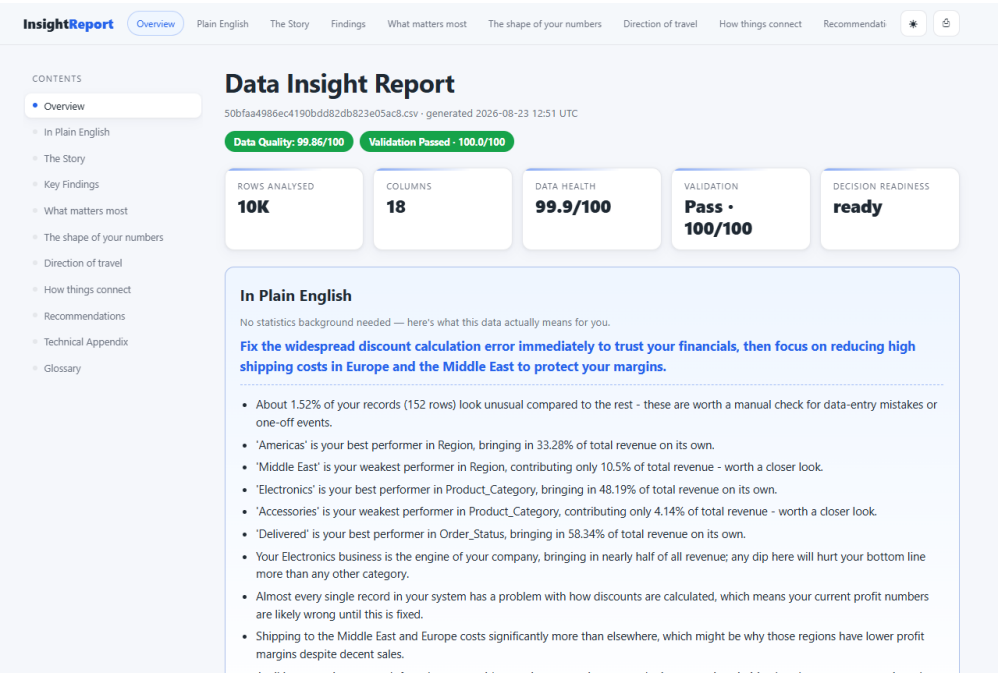

Figure 10-2. Sample output: Final report demo

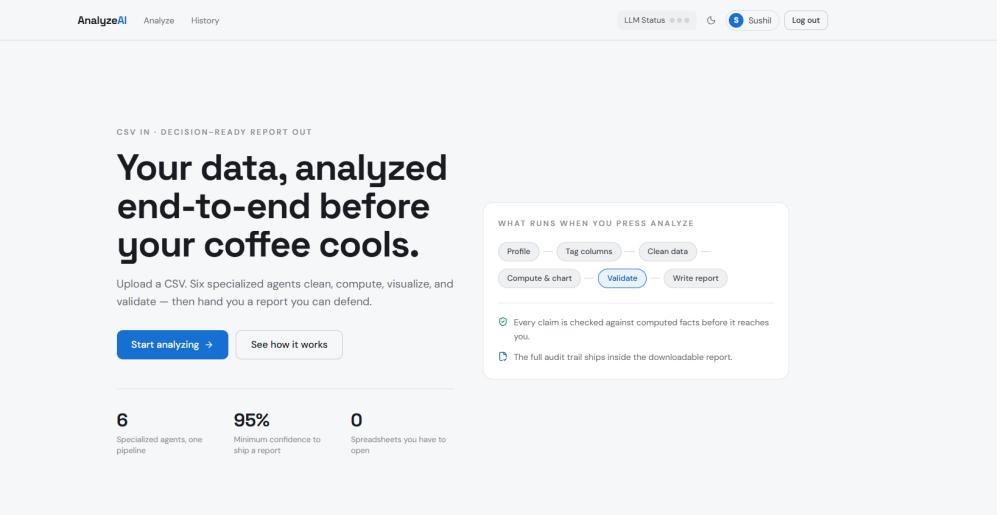

Figure 10-3. Sample output: Web application results dashboard

Appendix E: Agent Run Diagnostics Excerpt

Below is a representative run summary produced by the pipeline diagnostics:

```
{
  "run_id": "d6816f020542",
  "raw_shape": {"rows": 10000, "cols": 14},
  "cleaned_shape": [10000, 58],
  "preprocessing_profile": "strict",
  "dataset_domain": "finance_sales",
  "data_quality": 100.0,
  "validation_passed": true,
  "validation_score": 100.0,
  "cohen_kappa": 1.0,
  "flagged_issues": 0,
  "n_charts": 16,
  "narrative_source": "groq",
  "claims_grounding": {"claims_checked": 21,
     "claims_grounded": 21,
                      "claims_flagged": 0, "confidence": 1.0},
  "reliability": {
    "stage_confidence": {"agent1": 1.0, "agent3": 1.0,
       "agent4": 0.9,
                          "agent5": 1.0, "agent6": 0.95},
    "overall_confidence": 0.97,
    "decision_readiness": "ready"
  },
  "report_path":
     "outputs/reports/d6816f020542/insight_report.html"
}
```

Listing 10.1 Representative run summary from the pipeline diagnostics

REFERENCES

[1] Y. K. Dwivedi, L. Hughes, A. M. Baabdullah, et al., "So what if ChatGPT wrote it? Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy," *International Journal of Information Management*, vol. 71, 2023.

[2] OpenAI, "GPT-4 Technical Report," 2024.

[3] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, "LAMBDA: A Large Model Based Data Agent," arXiv preprint arXiv:2407.17535, 2024.

[4] Y. K. Liu, et al., "Summary of ChatGPT-Related Research and Perspective Towards the Future of Large Language Models," arXiv:2306.07209, 2023.

[5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in *Proceedings of ACL*, 2020.

[6] V. Dibia and C. Demiralp, "Data2Vis: Automatic Generation of Data Visualizations Using Sequence-to-Sequence Recurrent Neural Networks," *IEEE Computer Graphics and Applications*, vol. 39, no. 5, pp. 33–46, 2019.

[7] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. A. Hidalgo, "VizML: A Machine Learning Approach to Visualization Recommendation," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2019.

[8] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer, "Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco," *IEEE Transactions on Visualization and Computer Graphics*, 2018.

[9] Y. Luo, X. Qin, N. Tang, and G. Li, "DeepEye: Towards Automatic Data Visualization," in *IEEE International Conference on Data Engineering (ICDE)*, 2018.

[10] LangChain, "LangGraph: A Python Library for Building Stateful, Multi-Agent Applications," 2024.

[11] Q. Wu, G. Bansal, J. Zhang, et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," 2023.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[13] Qwen Team, “Qwen3-27B Technical Report,” 2024.

[14] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” in *Proceedings of EMNLP*, 2023.

[15] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” arXiv preprint arXiv:2104.09864, 2021.

[16] N. Shazeer, “GLU Variants Improve Transformer,” arXiv preprint arXiv:2002.05202, 2020.

[17] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Advances in Neural Information Processing Systems*, 2019.

[18] J. R. Landis and G. G. Koch, “The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[19] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020.